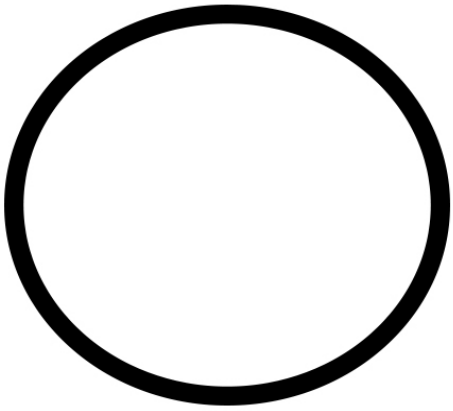


Caméléon

Notice

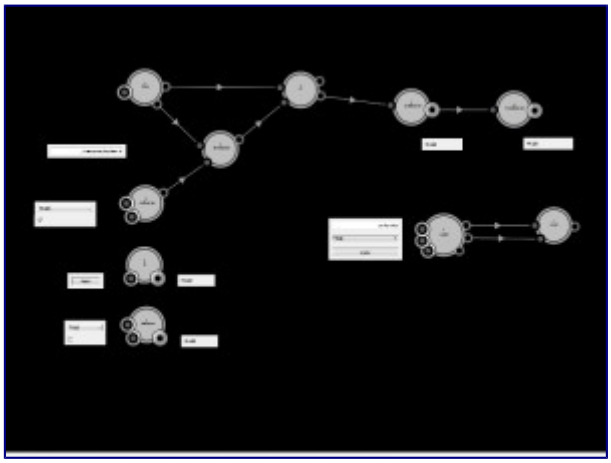
Document édité sous licence **Creative Common**

0.0.10 29/10/2016

http://www.shinoe.org

Caméléon & ... what is Caméléon ?

Caméléon is a **graphical language for fast prototyping** designed for data processing research, prototyping & calibration.
This is a **simple & intuitive** way to find and adjust high level functions compositions.
Its design allow you a **quick construction** of proof of concept and prototypes.
Caméléon Creator implements an **agile & robust language**, described here.



A composition

A prototype, or a composition, is a graphical network of elementary numerical components. You compose, prototype & calibrate with a simple drag & drop mechanism.

Caméléon & prerequisite skills

You must have a good experience in functionnal design & analysis to have a nice Caméléon experience.
You must have some BPMN & UML knowledges.
You must have a good abstraction capacity, a good understanding of information technologies & digital engineering.
You must love debian & digital technologies -is it a joke?.

Caméléon & prerequisite ressources

You need a computer (with a screen, a keyboard & a mouse).
You need a linux debian installed on it.
You need a developement environement ready to go (QT creator & QT4.8).

Caméléon & not required ressources

You don't need a mobile phone.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

You don't need a tablet.
You don't need a windows plateform.

System Requirement

We deliver Caméléon on a source code format, tested on debian or ubuntu with QT4 (4.8). You have to build your own instance in your own computer.

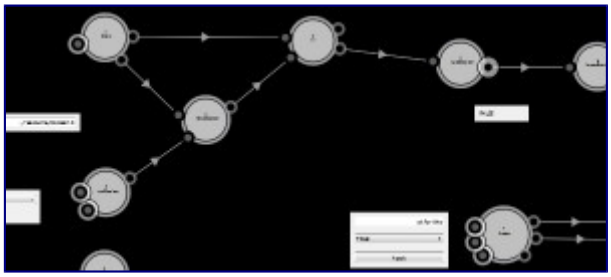
We are working on a QT5 version of Caméléon (version 2.1.x) planned to be delivered during 201X(?).

The system requirement will change regarding how big your data is. So be careful to test on little piece of data before showing us your gun!

Document édité sous licence **Creative Common**

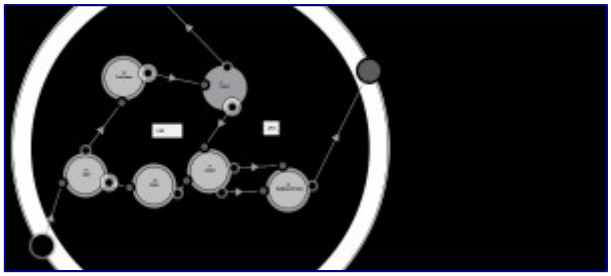
Macroscopic Features

Graphical Algorithm Editor



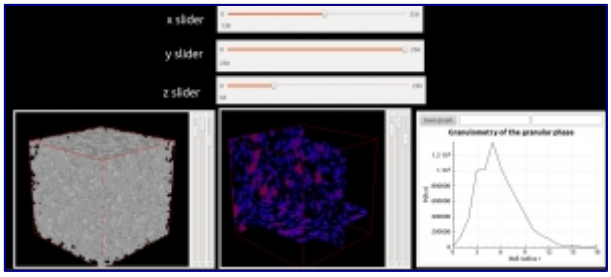
With a simple & intuitive drag & drop mechanism you create new prototypes; composition of elementary numerical components. Just with your mouse, you can create new combination, new ideas. Solution research is made intuitive, fast & easy.

Dynamic building



Proof of concepts, demonstrations can be made alive quickly thanks to Caméléon agility. Your design & presentation are dynamic, your audience can play easily with your work.

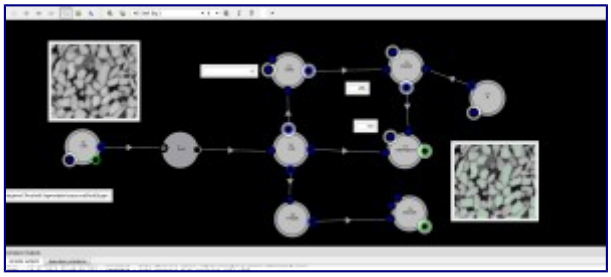
Real time calibration



When you have found the good elementary component combination, you can play with inputs parameters & see the result in real time. Find the right combination hasn’t been faster.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Integrate easily API, programs or CLI



You can integrate quickly new operators and fonctions from differents business fields.

Development kit



```
460     //Tool
461     //Format
462     this->registerOperator(new OperatorConvert1ByteImageGrid);
463     this->registerOperator(new OperatorConvert2ByteImageGrid);
464     this->registerOperator(new OperatorConvert4ByteImageGrid);
465     this->registerOperator(new OperatorConvertFloatImageGrid);
466     this->registerOperator(new OperatorConvertColor2GreyImageGrid);
467     this->registerOperator(new OperatorConvertGrey2ColorImageGrid);
468     this->registerOperator(new OperatorConvertColor2RGBImageGrid);
469     this->registerOperator(new OperatorConvertRGB2ColorImageGrid);
470     this->registerOperator(new OperatorConvertScalar2ComplexImageGri
471     this->registerOperator(new OperatorConvertComplex2ScalarImageGri
472
473     this->registerOperator(new OperatorImageGrid2ImageQT);
474     this->registerOperator(new OperatorImageQT2ImageGrid);
475     this->registerOperator(new OperatorConvertImage3DToVector);
476     this->registerOperator(new OperatorConvertVectorToImage3D);
```

The development kit allow you to add new algorithm written in C++.
See SCD & Connectivity dictionary, integrated with the development kit.

Document édité sous licence **Creative Common**

Install & Launch (linux debian or ubuntu)

The Caméléon SDK downloaded file is a tar file. Extract cameleon in any directory:

```
tar -zxvf ${FILE}
```

Build cameleon.pro project with a QT Creator and QT4 instance.

... then go to the cameleon directory.

On Linux/Unix, you need to make the file **deliverable/instance/cameleon.sh** executable in order to run it. You can either do this with your desktop's file manager or, at the command line, type:

```
chmod u+x cameleon.sh
```

You should now be able to execute the file. You can do this from the command line by typing:

```
./cameleon.sh
```



Document édité sous licence **Creative Common**

Quick Guide

Composer

The composer allows to create and connect graphically elementary components (operator, controls, patterns) to build new composition.



Figure 2 – The composer

* **Menu Bar** [Fig. 3]: The menu bar gives you access to management features
* **Player Bar** [Fig. 5]: Start, pause & stop the engine. You can either launch the engine step by step.
* **Mode Bar** [Fig. 6]: Three editing modes are available; composition (arrow icon), shape (square icon) & text (A icon).
* **Comment Bar** [Fig. 7]: Set up text & shape style & color.
* **Glossary Tree** [Fig. 4]: The glossary is like a component repository. You can create component from here.
* **Project Tree**: The project tree expose your prototype structure. You can navigate through sub processes & interfaces. You can add or delete interfaces.
* **Composition View**: The black center square. This is the place to create operators, controls & set up compositions.
* **Caméléon Outputs**: At the bottom, two tabs are available. The first, consol outputs, shows all Caméléon outputs. To set console outputs, go to windows > properties > project > Consol Outputs, set outputs to enable & select your outputs level for each logs. The second, Executions problems, shows all execution problmes actualy avaible. By clicking on them, the system highlight the erroneous component.

Drag & Drop everything

Drag & Drop component name from the glossary view to the workspace to create a component.
Drag & Drop a component to change its position.
Drag & Drop an operator connector to change its position.
Select several component and **Drag & Drop** those elements to change their positions.

Shortcuts

CTRL + Q + Mouse wheel: Zoom in / Zoom out
CTRL + A: Select all
Operator Selected + W: Show informations
DEL: Delete selected components

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Introduction

back to top

Concepts article is a good start to understand Caméléon underlying concepts.
All component construction features are based on a drag & drop mechanism.

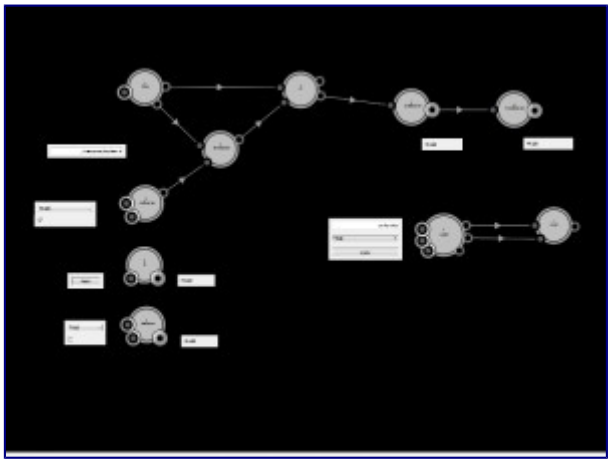


Figure 1 – A composition

Caméléon is a graphical algorithm editor. You can combine elementary components to create new compositions [Fig. 1].

Caméléon allow the creation of those components:
* **Operator**: Like a mathematical function, an operator, f, produces output data, y, from input data, x, as follows y = f(x). An operator (big center circle) has some input connector containing input data (little black circles including little colored or gray circles) and some output connector containing output data (little colored or gray circles including black circles).
* **Control**: A control is an elementary component to manipulate, set or view datas. It can be connected to operators inputs & outputs.
* **Pattern**: A pattern is a set of operators & controls aggregate in an elementary component. From a part of a composition you can create a sub composition, a sub process. A sub composition is reusable & sharable as pattern.
* **Text & Shape**: Text & Shape are used to structure & comment compositions.

Instance

back to top

A Caméléon instance is a directory holding all files, binaries needed to execute Caméléon Editor.
In the Caméléon, some directories and files are available:

```
/bin
/log
/lib
/ressource
/pattern
```

Document édité sous licence **Creative Common**

```
/project
instance.conf
[on linux] cameleon.sh
[on windows] cameleon.bat
```

The bin directory contains all binaries to execute Caméléon.
The log directory contains all Caméléon instance log files.
The lib project contains all external dictionaries installed in Caméléon.
The ressource directory contains all ressource file needed by Caméléon (images …).
The pattern directory is the directory where patterns are stored by Caméléon Creator.
The project directory contains example projects. This directory must be used to store your projects.
The instance.conf file is an XML configuration file for Caméléon.
The cameleon.sh script is used to launch Caméléon on Linux.
The cameleon.bat script is used to launch Caméléon on Windows.

Note: Sometimes Caméléon become unstable. We advice to make a copy of the instance.conf file. Then delete the instance.conf file and relaunch Caméléon.
Send-us the content of the instance.conf backup file using the contact form.

Composer

back to top

The composer allows to create and connect graphically elementary components (operator, controls, patterns) to build new composition.

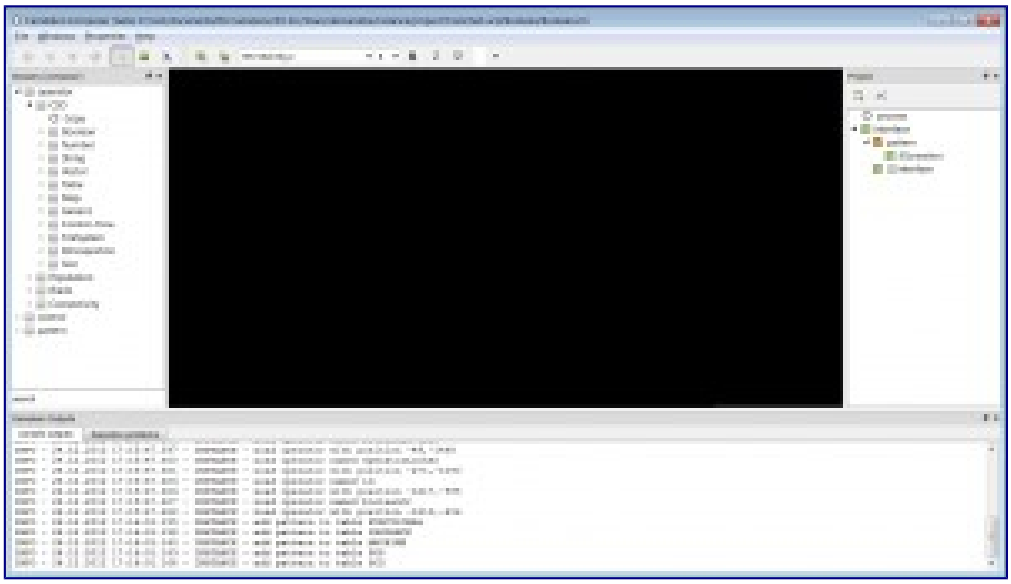


Figure 2 – The composer

* **Menu Bar** [Fig. 3]: The menu bar gives you access to management features
* **Player Bar** [Fig. 5]: Start, pause & stop the engine. You can either launch the engine step by step.
* **Mode Bar** [Fig. 6]: Three editing modes are available; composition (arrow icon), shape (square

icon) & text (A icon).
* **Comment Bar** [Fig. 7]: Set up text & shape style & color.
* **Glossary Tree** [Fig. 4]: The glossary is like a component repository. You can create component from here.
* **Project Tree**: The project tree expose your prototype structure. You can navigate through sub processes & interfaces. You can add or delete interfaces.
* **Composition View**: The black center square. This is the place to create operators, controls & set up compositions.
* **Caméléon Outputs**: At the bottom, two tabs are available. The first, consol outputs, shows all Caméléon outputs. To set console outputs, go to windows > properties > project > Consol Outputs, set outputs to enable & select your outputs level for each logs. The second, Executions problems, shows all execution problmes actualy avaible. By clicking on them, the system highlight the erroneous component.

Menu

Menu gives access to saved file manager, properties etc.

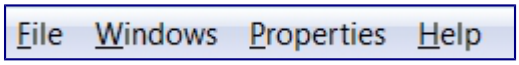


Figure 3 – Menu Bar

* **File**: Save & Load Caméléon *.cm files.
* **Windows**: Hide & Show windows.
* **Properties**: Go to project or instance properties dialogs.
* **Help**: Go to Caméléon web site to find information.

Glossary

The glossary gives access to all components which can be created by Caméléon Creator.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

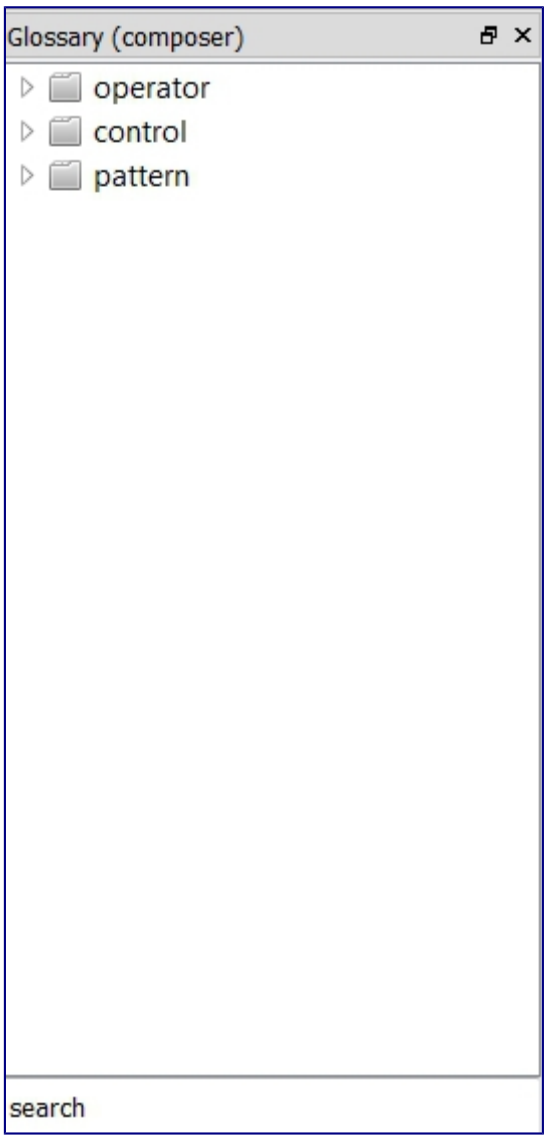


Figure 4 – Glossary

The glossary is a tree list view which exposed all components which can be created with your Caméléon software.
The glossary is devided in three parts:
* **Operator**: Set of all available algorithms. Directories under the Operator directory are named glossary.
* **Control**: Set of all available controls. Directories under the Control directory are named glossary.
* **Pattern**: Set of all available user created patterns.

Browse the glossary:
1- Open a component directory,
2- Little **O** icons identify components which can be created. Drag & Drop it to the composition view to create a component.

Search an operator:
1- Click on the search edit box,
2- Write a key word to search,
3- The search tool highlight corresponding components.

Player Bar

The player bar allow you to launch or stop the Caméléon engine.

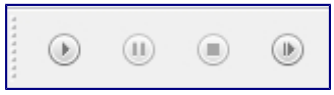


Figure 5 – Player Bar

* **Run**: to launch the composition execution,
* **Pause**: to pause the execution,
* **Stop**: to stop the execution,
* **Next step**: in pause mode, allow you to process one next operator.

Mode Bar

The mode bar allows to change the edition mode.



Figure 6 – Mode Bar

* **Composition Mode**: The mode to compose with operators & controls,
* **Shape Mode**: The mode to draw shapes,
* **Text Mode**: The mode to write text (comments …).

Comment Bar

The comment bar allow to manage text style & shape colors.



Figure 7 – Comment Bar

Those actions has effect on selected texts. Multiple selection is allowed.
* **Police selector**: Change selected text police,
* **Size selector**: Change selected text size,
* **Bold button**: Change selected text to Bold,
* **Italic button**: Change selected text to italic,
* **Underline button**: Underline selected text,

This action has effect on selected texts & shape. Multiple selection is allowed.
* Color selector: Change selected text or shape color.

Bring & Send

The bring & send bar allows to manage component Z axis.



Figure 8 – Bring & Send

Those actions has effect on selected components. Multiple selection is allowed.
* **Bring to front**: Bring selected components to front,
* **Send to back**: Send selected components to back.

File Menu

File menu allows to manage load & save project.

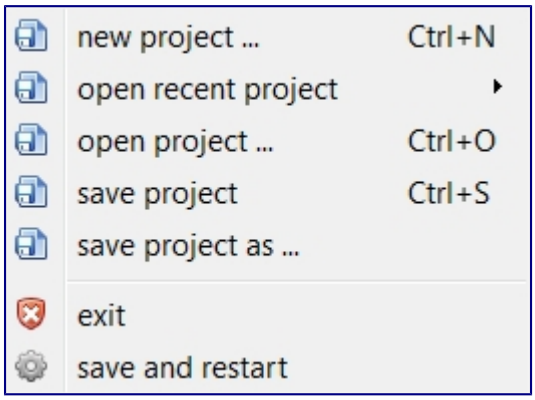


Figure 12 – File Menu

* **new Project …**: Create a new project,
* **open recent project**: Open a recent project list. Select a project to open it,
* **open project …**: Open a project on your computer,
* **save project**: Save a project on your computer,
* **save project as …**: Save a project in a specific directory,
* **exit**: Close Caméléon,
* **save and restart**: Save and restart Caméléon.

Windows Menu

The windows menu allows to manage windows visibility.

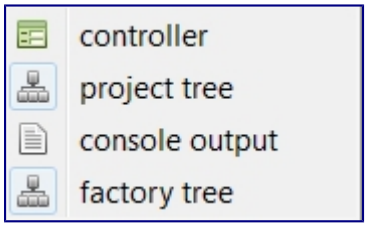


Figure 9 – Windows Menu

* **controller**: Show/Hide the controller window,
* **project tree**: Show/Hide project tree,
* **console output**: Show/Hide console outputs,
* **glossary tree**: Show/Hide glossary tree.

Document édité sous licence **Creative Common**

Properties Menu

The properties menu allows to manage instance & project properties.

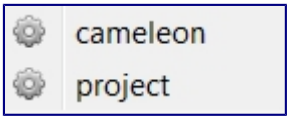


Figure 10 – Properties Menu

* **cameleon**: Show Caméléon properties dialog,
* **project**: Show project properties dialog.

Help Menu

Help menu gives access to support & documentation web links.

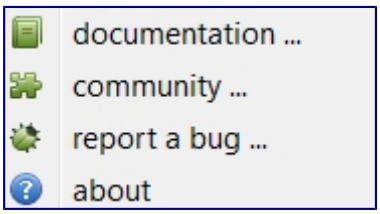


Figure 11 – Help Menu

* **documentation …**: Open online documentation in a web browser,
* **community …**: Open community links in a web browser,
* **report a bug …**: Open the online contact form,
* **about**: Open the about dialog (Caméléon version & licence are available from here).

Operator & Connector

back to top

Create & move an operator

Document édité sous licence **Creative Common**

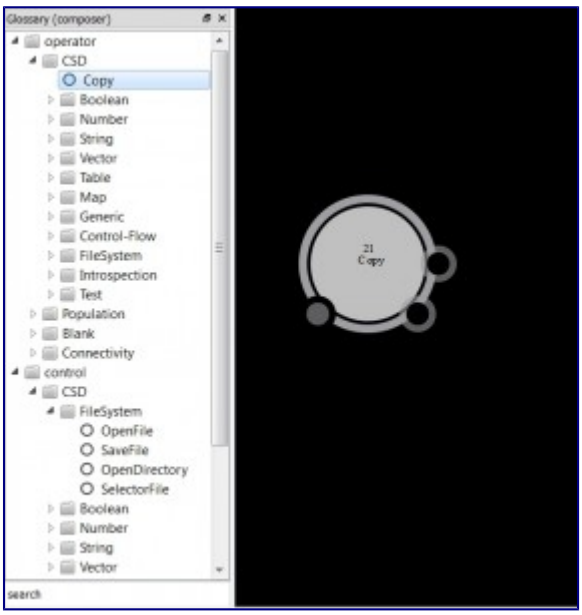
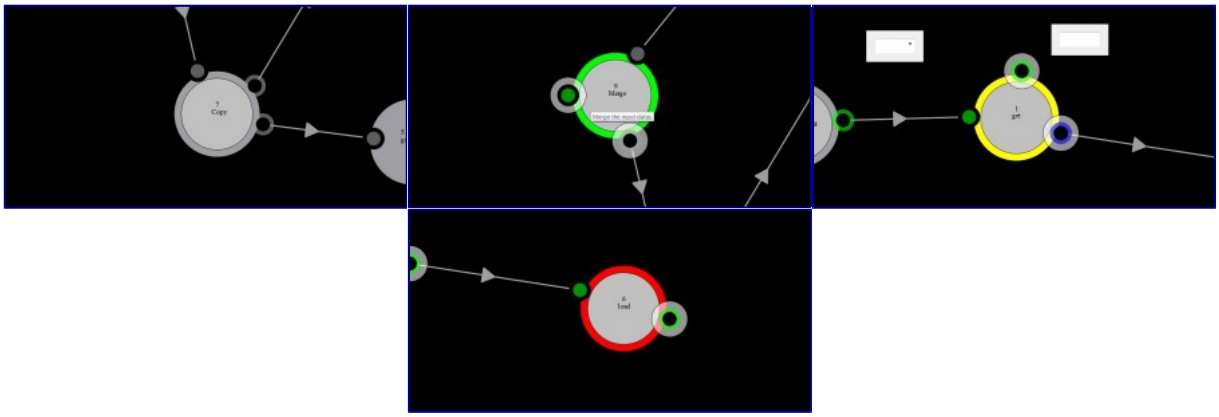


Figure 12 – Create an operator

1- Identify the operator you want to create by browsing or searching the glossary,
2- Operators are available behind the “operator” directory and are identified by a **O** icon,
3- Drag the operator’s name from the glossay and drop it in the composition view,
4- It creates an operator at the dropping position,
5- You can drag & drop it on the composition view to change its position (also works in multi selection).

Operator & Connector Statuses

The operator status gives information about its execution.


Figure 19 – Operator statuses

* **Runnable**: Green color. The operator can be executed.
* **Running**: Yellow color. The operator is being executed.
* **Not Runnable**: Gray color. The operator can not be executed by the CVM.
* **Error**: red color. An error occurred on this operator.

Connector status gives information about the connector’s data.

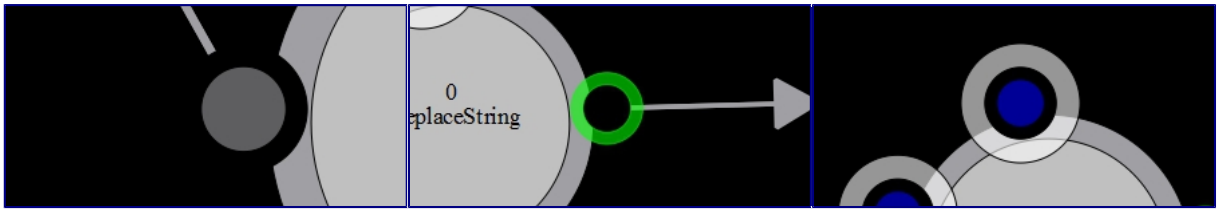


Figure 20 – Connector statuses

* **Empty**: Gray color. No data available in the connector.
* **New**: Green color. A new data is available in the connector.
* **Old**: Blue color. An old data -means already used by an operator, is available in the connector.

Change connector position

Organize simply the composition changing connector's position.

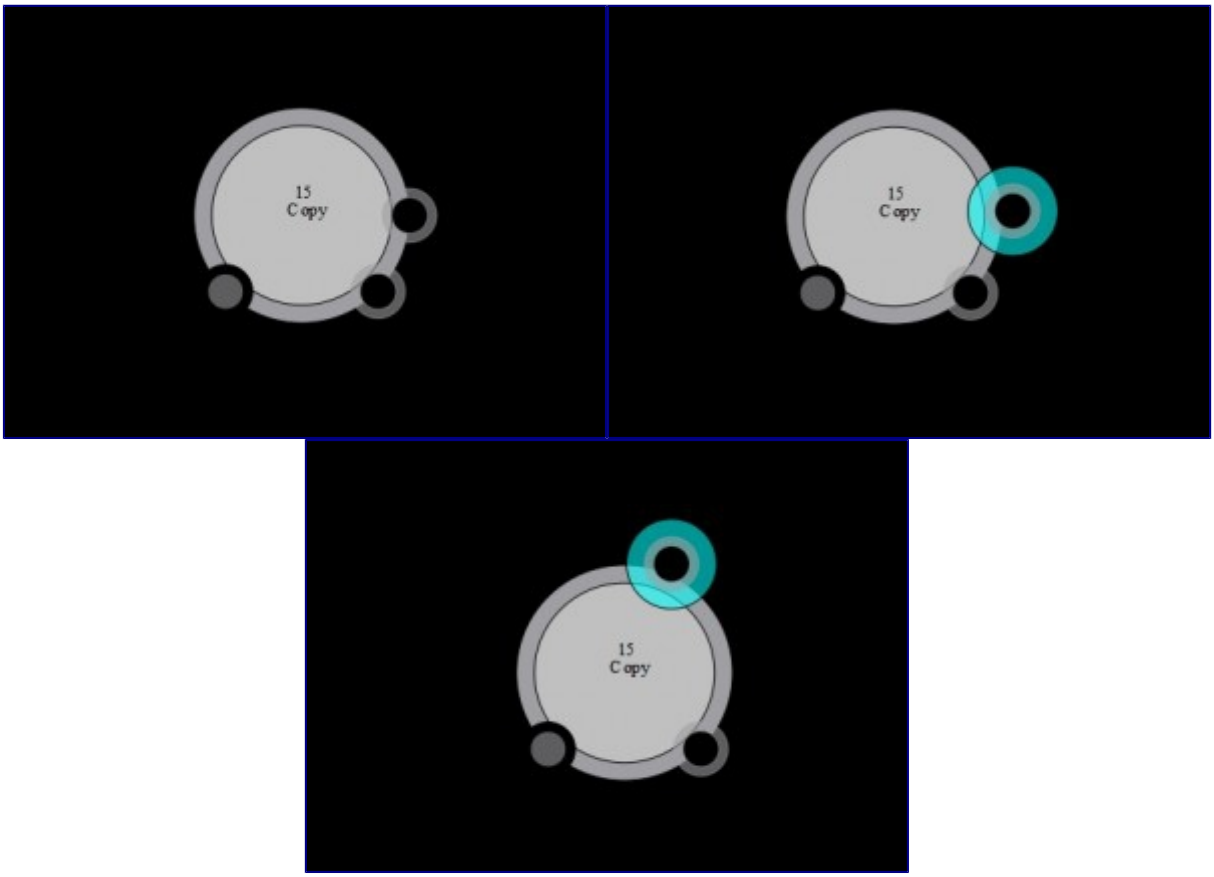


Figure 13 – Change connector position

1- Drag a connector to another position,
2- Be careful to keep your mouse into the connector circle,
3- Drop the mouse to set the connector position.

Document édité sous licence **Creative Common**

Show operator information

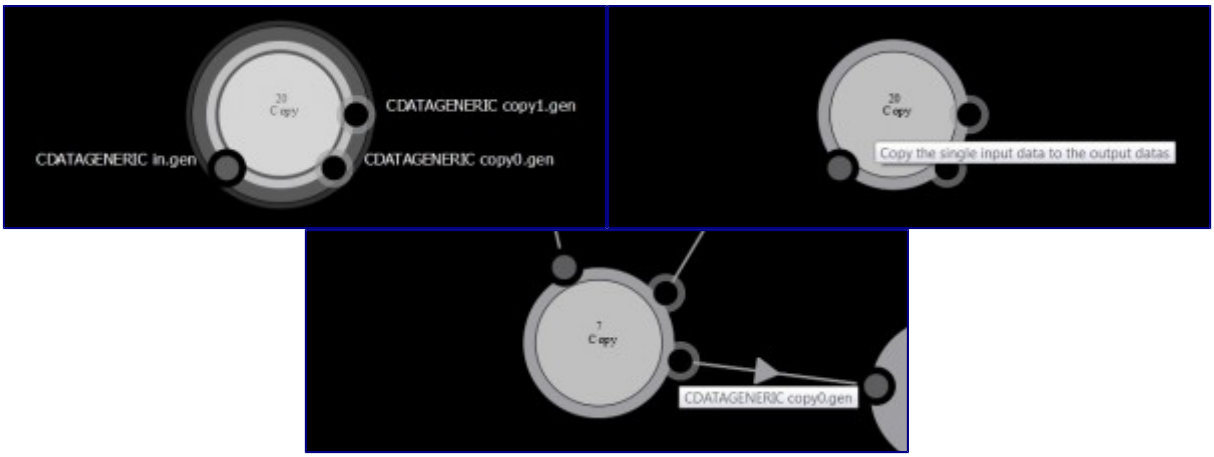


Figure 14 – Show operator information

Hoover your mouse on an operator or a connector to show information. Select one or more component, press the ‘W’ key to show more informations.

Operator Menu

Right Click on an operator to show the menu.

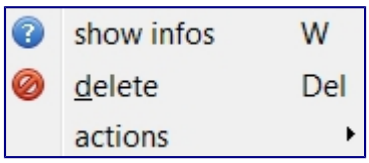


Figure 15 – Operator Menu

* **show infos**: As ‘W’ key press, it shows operator description,
* **delete**: delete the operator.
* **(optional) actions**: can expose operator specific actions.

Connection

Document édité sous licence **Creative Common**

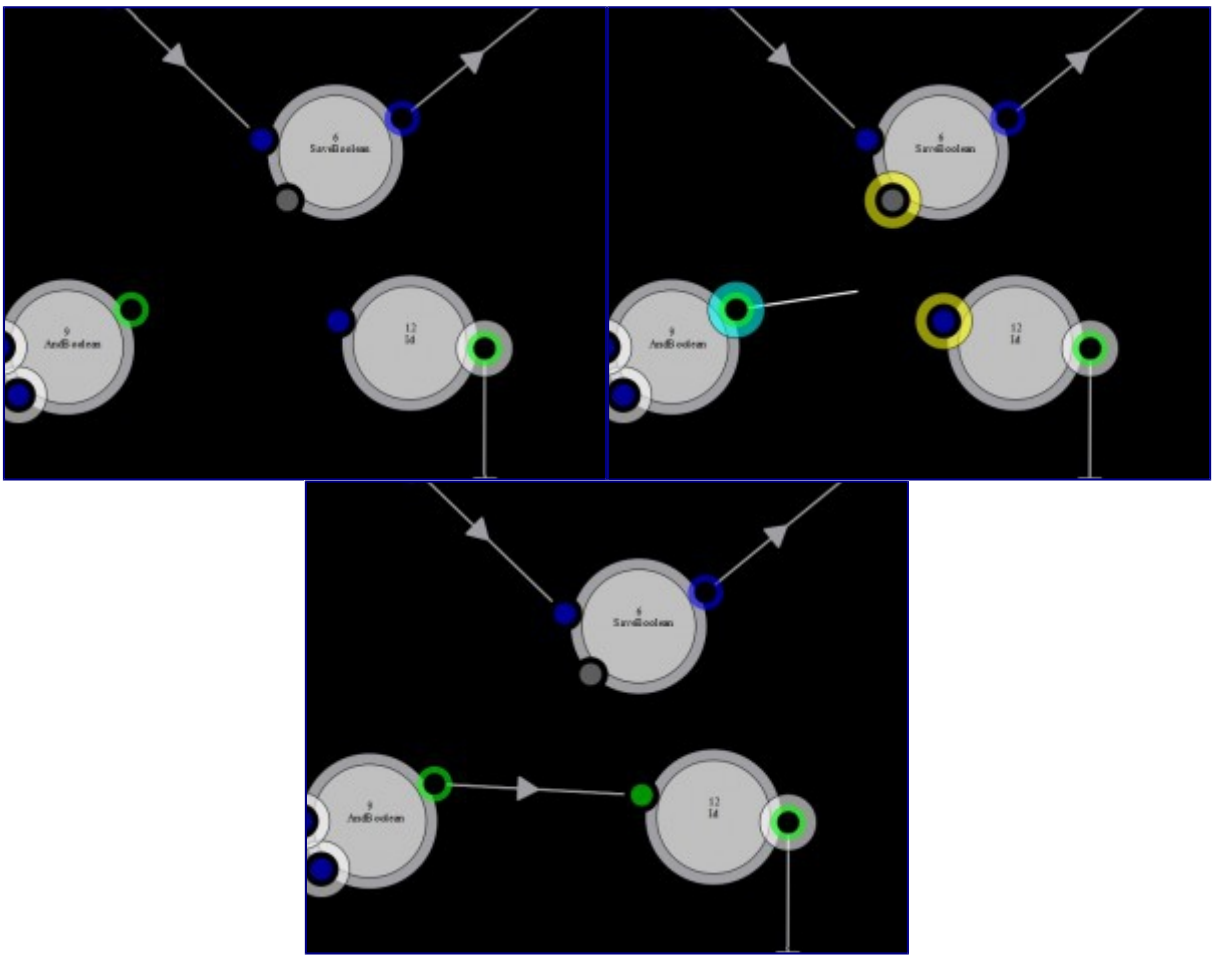


Figure 18 – Connection

1- Drag a connector from its position to somewhere on the composition view,
2- It shows compatible connectors (identified with yellow circles),
3- Drop on a compatible connector to connect.

After a connection, when a data will be available on output connector, it will be automatically dumped into input connector.

Note: Some datas are shared by reference or by value. The next 2.0.8 version will expose this characteristic more clearly.

Connector Menu

Right click on a connector to show the connector contextual menu.

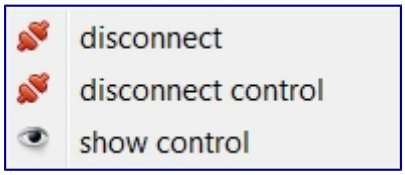


Figure 21 – Connector Menu

* **disconnect**: disconnect the connector,
* **disconnect control**: disconnect the connector from a control connector,

* **show control**: show the control connector connected to this connector.

Multi Selection

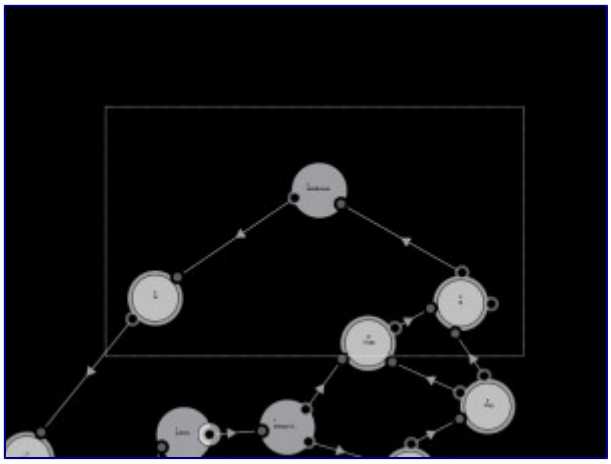


Figure 16 – Multi Selection

1- Drag the mouse from an empty space on the composition view to somewhere,
2- It draws a selection square,
3- Drop somewhere,
4- It selects components which are under the selection square.

After multi-selection, click right somewhee to show the multi-selection contextual menu.

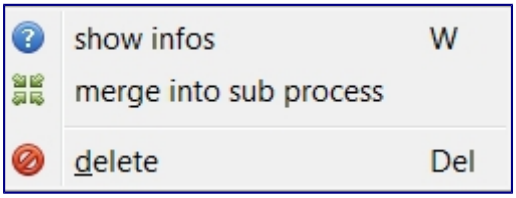


Figure 17 – Multi Selection Menu

* **show infos**: show information about selected components,
* **merge into sub process**: merge selected component into a new sub process,
* **delete**: delete selected components.

Controls

back to top

Create control

Document édité sous licence **Creative Common**

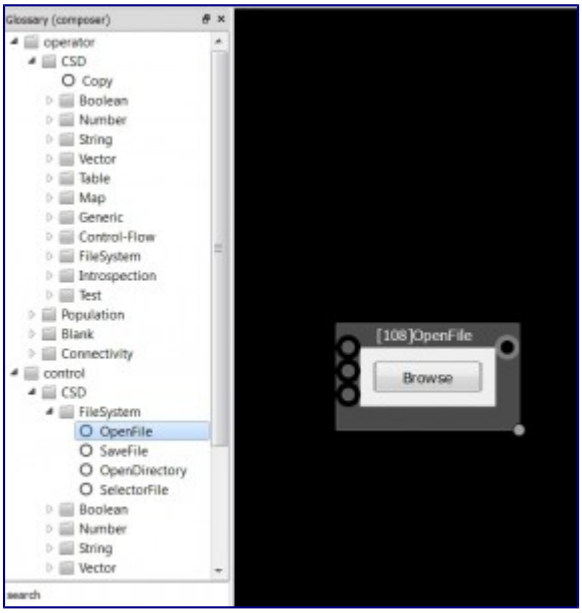


Figure 22 – Create control

1- Identify the control you want to create by browsing or searching the glossary,
2- Controls are available behind the “control” directory and are identified by a **O** icon,
3- Drag the control’s name from the glossay and drop it in the composition view,
4- It creates a control at the dropping position,
5- You can drag & drop it on the composition view to change its position (also works in multi selection).

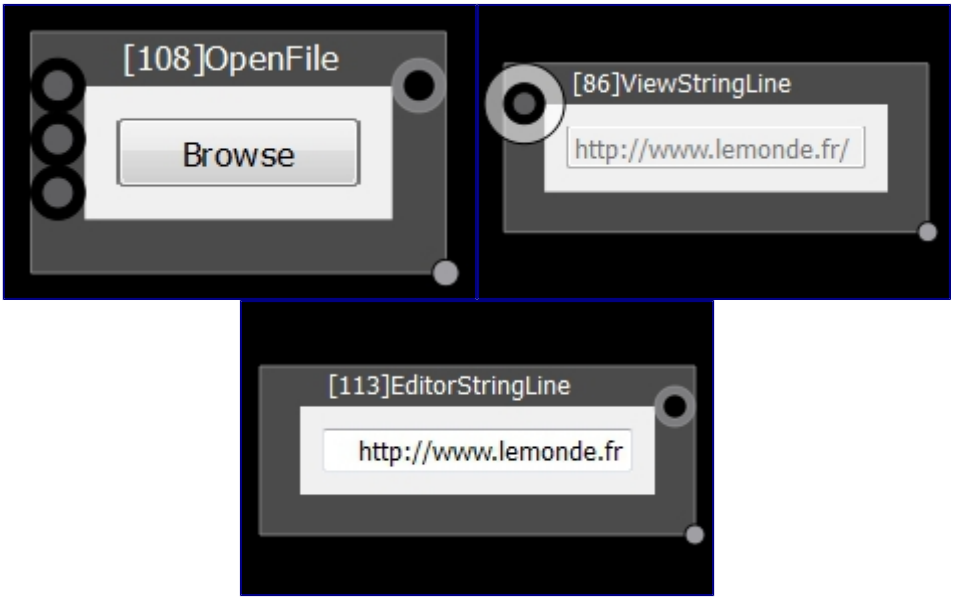


Figure 23 – Different kind of controls

Some controls are just data view, other can just edit & set data (often by pressing enter after edition). An other kind of control can edit datas using inputs datas.

You can resize the control by drag & dropping the little bottom right gray circle.

Control Menu

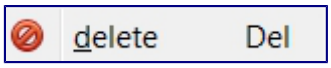


Figure 25 – Control Menu

Delete selected controls.

Control Connection

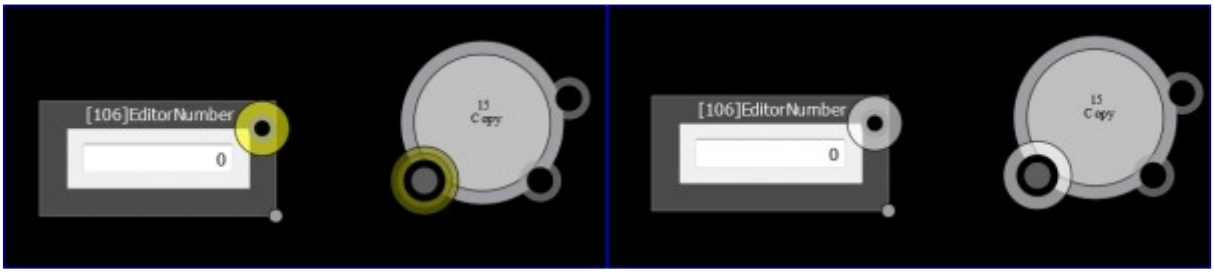


Figure 24 – Control connection

1- Click on a control connector,
2- See highlighted compatible connectors,
3- Click on one compatible connectors to connect,
4- Connector connected to control are identified with a white circle.

Show control Connection

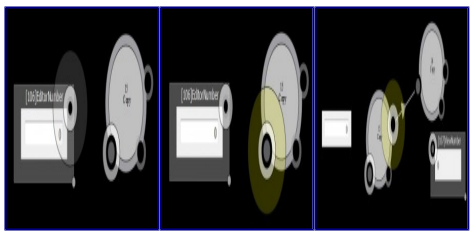


Figure 26 – Show connection

1- Double click on a connector connected to a control,
2- It highlights the connected connector.

Outputs operator connector can be connected to one operator connector and one control connector. In this case, the output datas can be used by an operator and, also, be viewed by a control.

Note: Be careful to data by reference.

Sub-Composition

back to top

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Create a sub composition

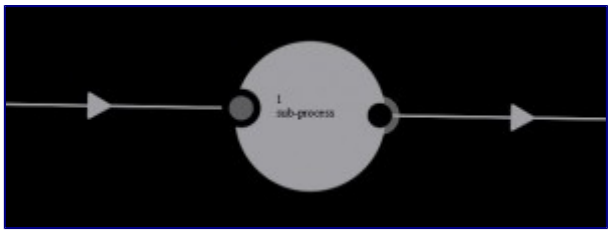


Figure 27 – A sub composition

1- Select more than one component (operator, control, text or shape),
2- Click left somewhere,
3- A multi selection menu appears,
3- Click on “merge into sub process”,
4- It creates a sub process, a gray circle.

Browse sub process

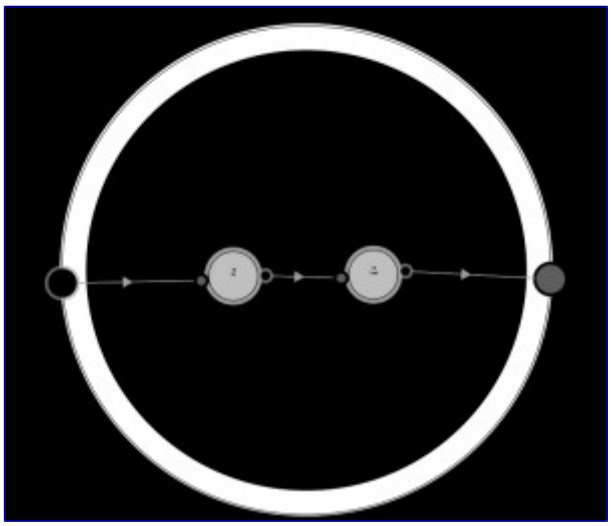


Figure 28 – Browse a sub process

1- Double click on a sub process to get into,
2- Double click on the sub process background to go to the upscale level.

Expose a connector

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

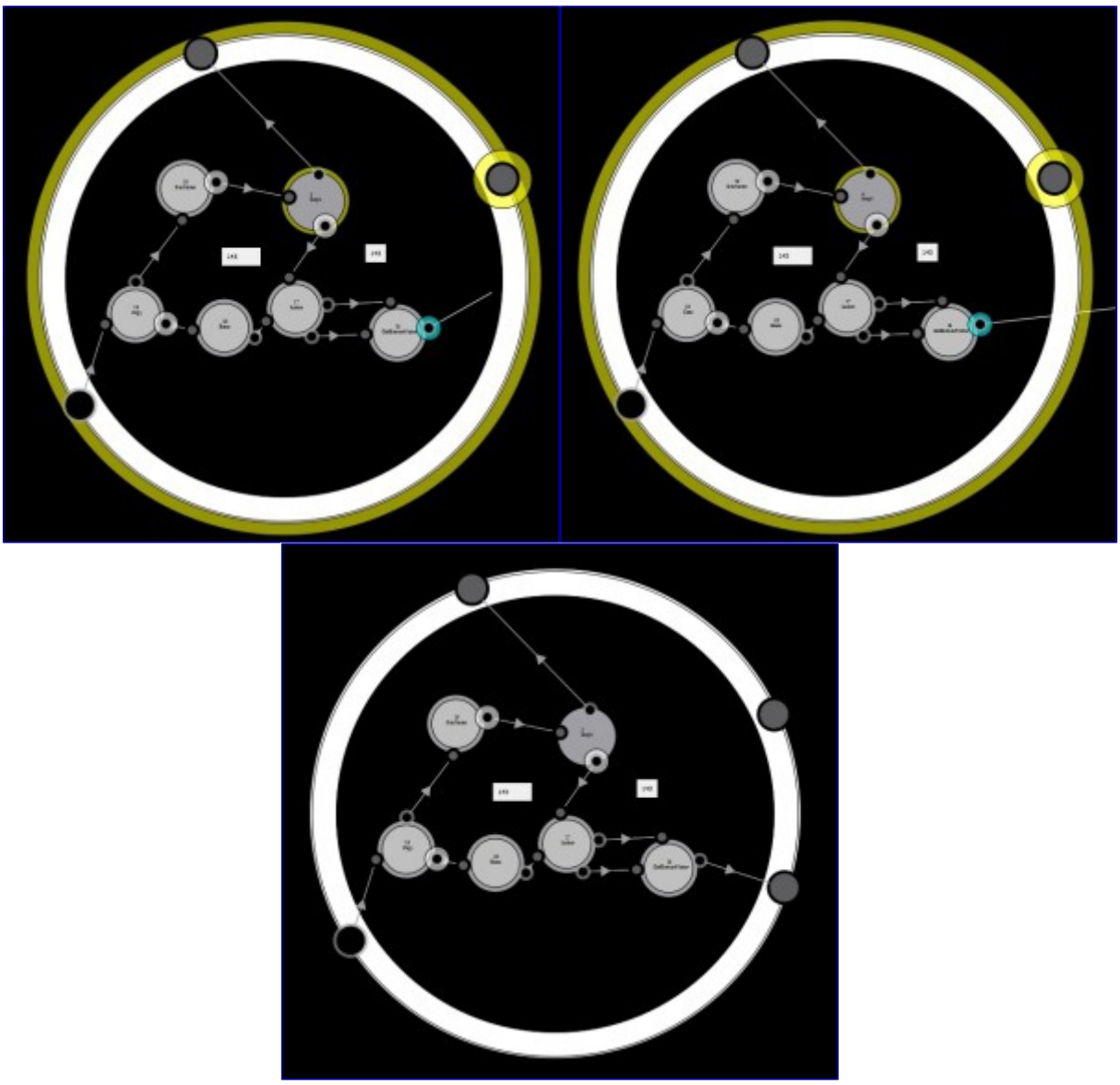


Figure 29 – Expose a connector

1- Drag a connector,
2- See highlighted compatible connectors and the sub composition highlighted,
3- Drop your mouse on the sub composition external background,
4- It create a new connector bridge on the sub composition and connect the connector to it.

Expose a connector (alternative)

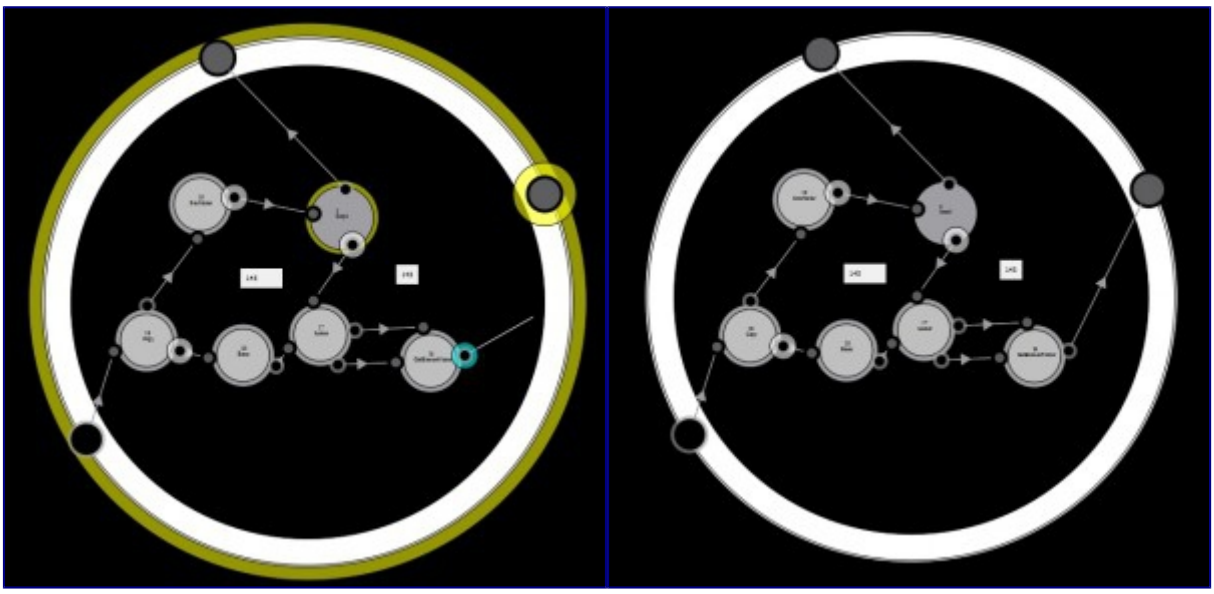


Figure 55 – Expose a connector (alternative)

1- In a sub composition, drag a connector,
2- Drop it on a sub composition connector,
3- Connectors are connected.

Connect to sub process

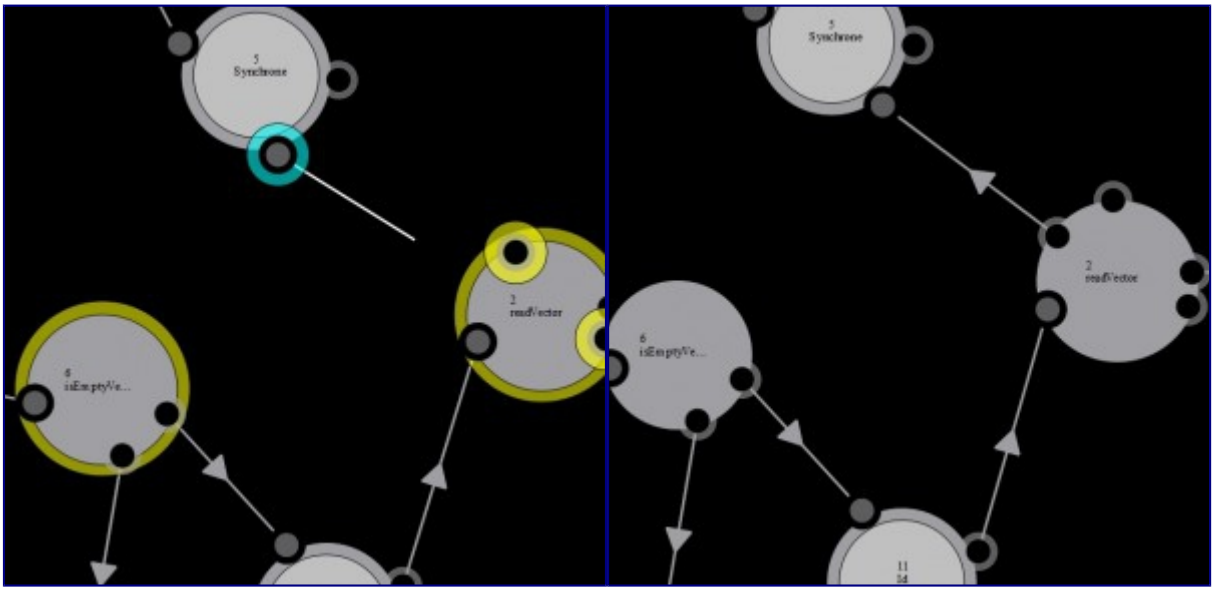


Figure 56 – Connect to sub process

1- At upper scale, drag a connector,
2- Drop the mouse on a sub composition,
3- It create a new sub composition bridge & connect the dropped connector to it.

Sub Process Menu

Click right on a sub composition to show the sub composition Menu.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org



Figure 30 – Sub Process Menu

* **show infos**: Show information about the sub process (those information are editable through the properties dialog),
* **save as a pattern**: Save this sub composition as a reusable pattern (see the pattern directory in the glossary),
* **properties …**: Show the properties dialog,
* **delete**: delete the sub composition,
* **unmerge**: restore the composition without the sub process.

Sub Process Properties

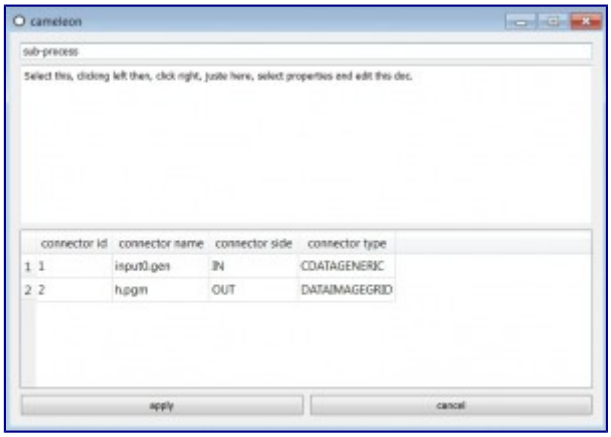


Figure 31 – Sub Process Properties Dialog

* **First edit box**: This is the sub composition name field. You can edit the name here,
* **Second edit box**: This is a description field. You can write the sub composition description here,
* **Connector List**: This is the list of all sub composition connectors. The name field is editable,
* **Apply button**: Click on this button to apply your change,
* **Cancel button**: Click on this button to discard your change.

Save as a pattern

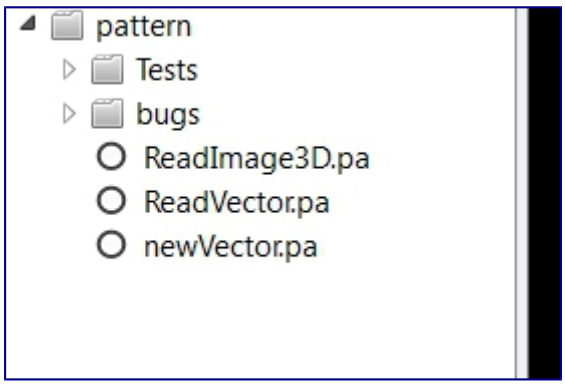


Figure 57 – Save as a pattern

When you click on save as pattern on the sub composition menu, it create a new entry under the “pattern” directory in the glossary.

Create a pattern

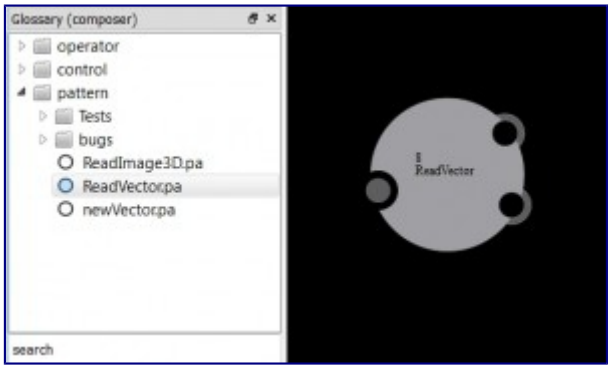


Figure 35 – Create a pattern

1- Identify the pattern you want to create by browsing or searching the glossary,
2- Operators are available behind the “pattern” directory and are identified by a **O** icon,
3- Drag the pattern’s name from the glossay and drop it in the composition view,
4- It creates a sub composition at the dropping position,
5- You can drag & drop it on the composition view to change its position (also works in multi selection).

Like for operator, you can change a connector’s position by drag & dropping it.

Create a pattern directory

Click right on the pattern directory or on an other directory under pattern and click on add directory. Select a name and it will create a new directory under the selected clicked right directory.

For sub directories or patterns,
1- click right on a sub directory or a pattern,
2- select rename menu to rename the directory or the pattern,
3- select delete menu to delete the directory or the pattern.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Organize patterns

Drag & drop pattern names or directories to organize patterns.

Text & Shape

back to top

Create a shape

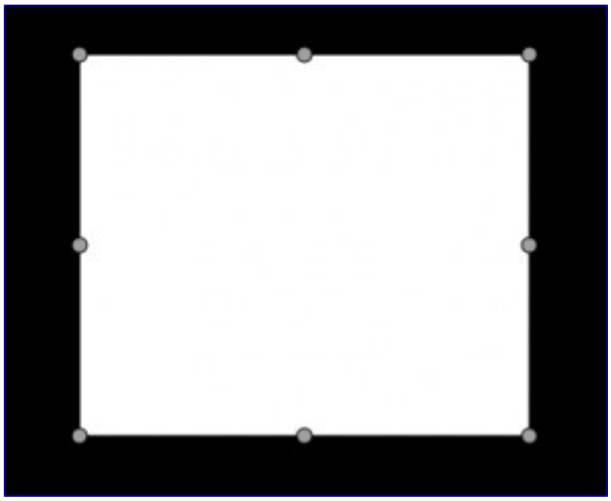


Figure 32 – Create a shape

1- Click on the shape mode button,
2- Drag your mouse from somewhere on the composition view,
3- It draws like a selection square,
4- Drop your mouse to define the size of the shape,
5- Click on the composition mode button,
6- By drag & drop, move the shape position,
7- By drag & dropping little circle on the bottom right, you can change the shape size.

Write a text



Figure 33 – Write a text

1- Click on the text mode button,
2- Right click somewhere on the composition view,
3- Write your text,

Document édité sous licence **Creative Common**

4- Click on the composition mode button,
6- By drag & drop, move the text position,

Shape & Text Menu

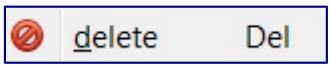


Figure 39 – Shape & Text Menu

Delete selected shape and text.

Navigation

back to top

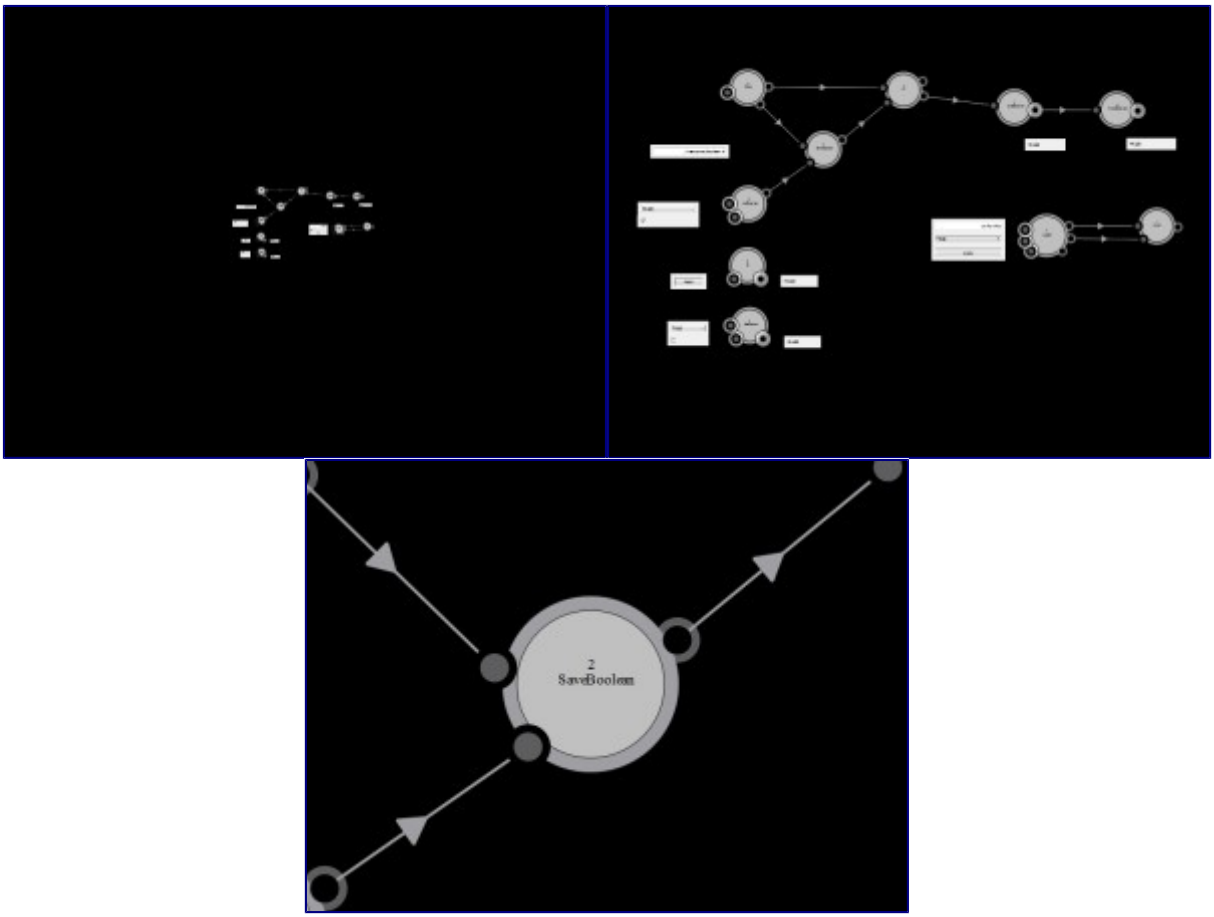


Figure 34 – Zoom in & out

* Hold the Q key on the keyboard and drag & drop your mouse to change the composition view position,
* Hold the Q key on the keyboard and use the wheel of the mouse to zoom in & zoom out.

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Controller

back to top

The controller is used to create functional interface prototype usable without using the composer.

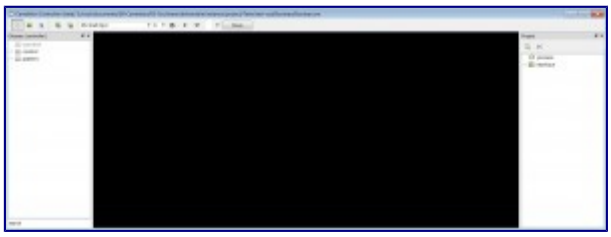


Figure 40 – Controller

* **Player Bar** [Fig. 5]: Start, pause & stop the engine. You can either launch the engine step by step.
* **Mode Bar** [Fig. 6]: Three editing modes are available; composition (arrow icon), shape (square icon) & text (A icon).
* **Comment Bar** [Fig. 7]: Set up text & shape style & color.
* **Glossary Tree** [Fig. 4]: The glossary is like a component repository. You can create component from here.
* **Project Tree**: The project tree expose your prototype structure. You can navigate through sub processes & interfaces. You can add or delete interfaces.
* **Interface View**: The black center square. This is the place to create operators, controls & set up compositions.

Controller's glossary

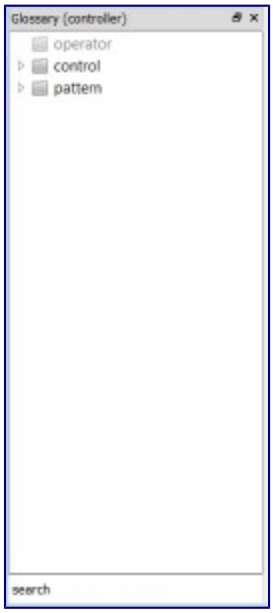


Figure 40 – Controller's glossary

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

The controller’s glossary is a tree list view which exposed all components which can be created with your the controller of the Caméléon software.
* **Control**: Set of all available controls.

Create a control

Figure 42 – Create a control

1- Identify the control you want to create by browsing or searching the glossary,
2- Controls are available behind the “control” directory and are identified by a **O** icon,
3- Drag the control’s name from the glossay and drop it in the composition view,
4- It creates a control at the dropping position,
5- You can drag & drop it on the composition view to change its position (also works in multi selection).

Control Menu

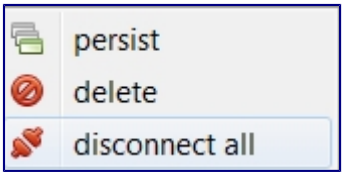
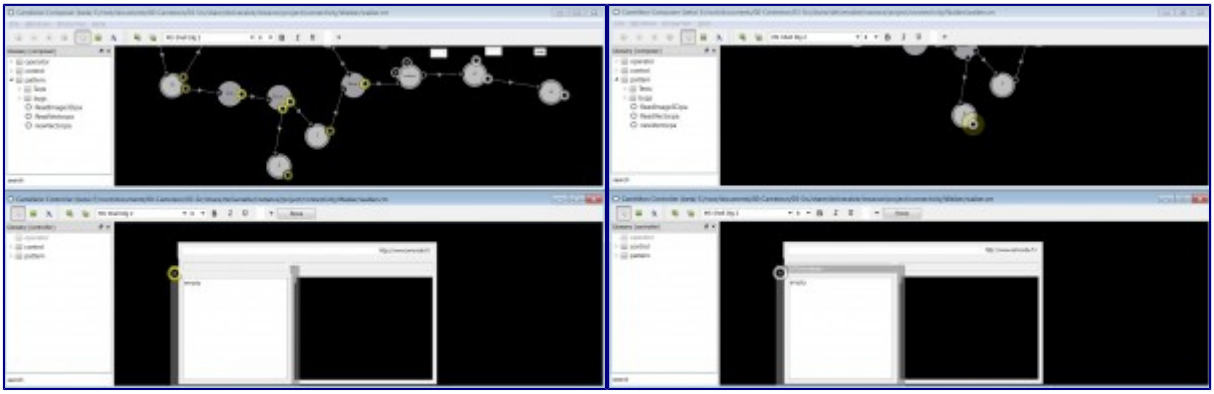


Figure 41 – Control Menu

* persist: If active, the control will be available at the same position in all interface (see create an interface),
* delete: Delete the control,
* disconnect all: Disconnect all control connectors.

Connect a controller control to an operator


Figure 43 – Controller control connection

1- Click on a control connector on the controller window,
2- See highlighted compatible connectors on the composer window,
3- Click on one compatible connectors to connect,
4- Connector connected to control are identified with a white circle.

Document édité sous licence **Creative Common**

Note: Be careful to find the right position & size for both controller window & composer window. We must see both the to composition view & the interface view to connect correctly a control on the controller side to an operator on the composer side.

You can double click on the connected connector to see highlighted connected connector.

Project

back to top

The project tree (right part of the composer or the controller) can be shown or hided using menu **windows** > **project tree** action.



Figure 44 – Project #1

The project tree allows to browse composer sub compositions or controller interfaces. It allows to create sub processes or interfaces.

Browse processes

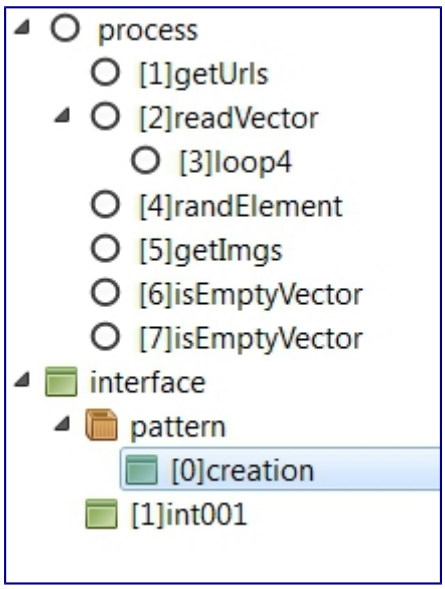


Figure 55 – Project #2

Double click on a process (**O** icons) to show the corresponding process on the composer.
The top process (named “process”) goes to the main top composition.

Browse interfaces

Double click on a process (**Square green** icons) to show the corresponding interface on the controller.
The interface under pattern (named “creation”) goes to the main interface.

Create an interface


Figure 54 – Creation icons

Click on the **square green plus** icon to create a new interface. A pop-up to set up its name will appear.

Create a sub process

Click on the **blue/green graph node** icon to create a new sub process. A pop-up to set up its name will appear.
The sub composition is created at the center of the current composition/sub composition.

Project directory

A project directory must be created by user for each new project. The save as feature must be used in this directory. For instance, we save as a new project named “project_name” in an empty project_name directory. The directory is composed of those sub-directories & files:

```
project_name/log
project_name/ressource
```

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

```
project_name/target
project_name/tmp
project_name/project_name.cm
```

The log directory contain the cameleon.log file (and maybe others). In case of a crash, this file can be usefull to analyse the problem. Log output is configurable through the Project Properties Logs dialog.
The ressource diretory is used to put all datas file used by the project. To share a project, it's usefull to have a place to put those data sources (images, raw files, videos etc.).
The target directory is *deprecated*.
The tmp directory contain data file copy produced by the composition.
The project_name.cm file is the XML file descriptor of the project (reference to operators, data, controls, interfaces & sub compositions).

Properties

back to top

Cameleon Properties Options

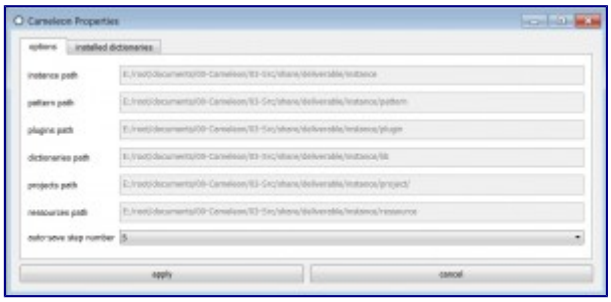


Figure 45 – Cameleon Properties Options

* Instance path: The instance path,
* Pattern path: Patterns are stored here,
* Plugins path: deprecated,
* Dictionaries path: dictionaries shared libraries are stored here,
* Projects path: project must be saved here,
* Ressources path: unstable,
* Auto-save step number: Number of action to launch an auto-save.

Caméléon Properties Dictionaries

Document édité sous licence **Creative Common**

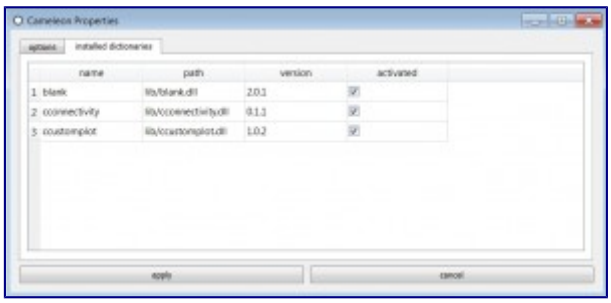


Figure 46 – Caméléon Properties Dictionaries

Table:
* name: The dictionary name,
* path: The dictionary path,
* version: (error) The version or an error message if miss loaded,
* activated: enable or disable the dictionary (need a Caméléon restart).

Project Properties Options

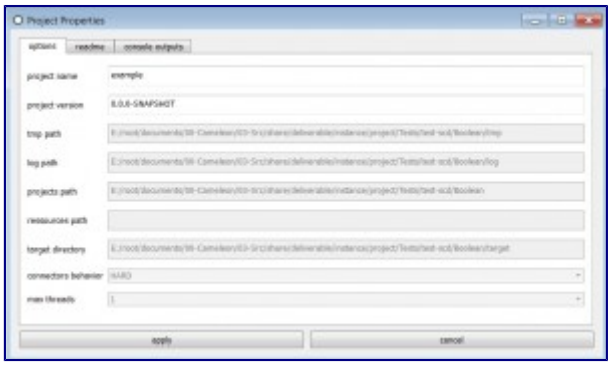


Figure 47 – Project Properties Options

* project name
* project version
* tmp path
* log path
* projects path
* ressources path
* target directory
* connectors behavior: debug tool
* max threads: disabled because unstable

Project Readme

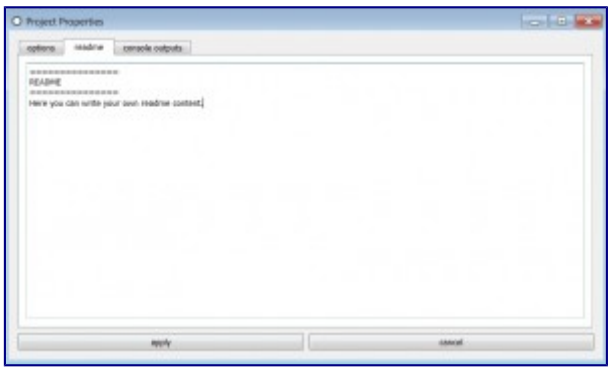


Figure 48 – Project Readme

The project readme editable text box.

Project Properties Logs

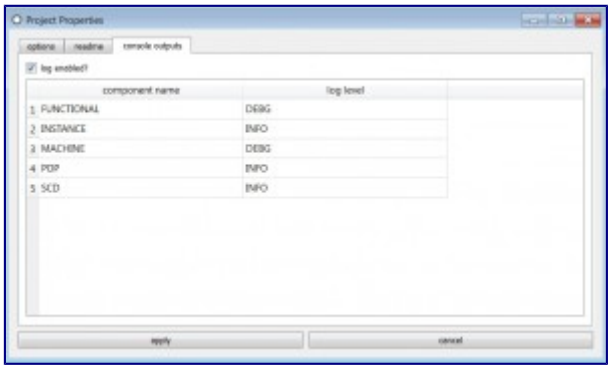


Figure 49 – Project Properties Logs

Firet section:
* log enabled check box: Enable or disable all logs.

Table:
* component name: Log component are autonomous log which focus on specifics parts of the Caméléon execution.
* log level: You can select your log level or shutdown a log component.

Sub process properties

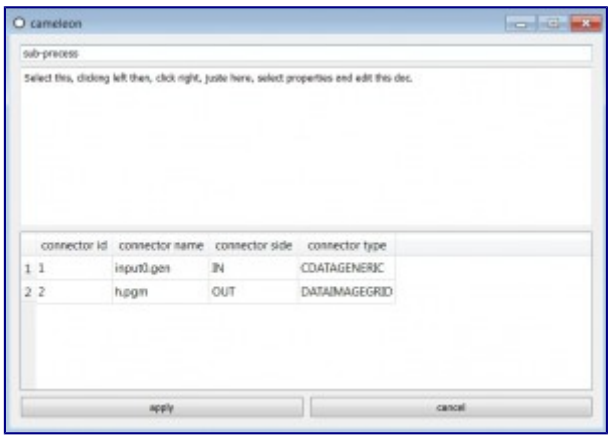


Figure 50 – Sub process properties

* First edit box: This is the sub composition name field. You can edit the name here,
* Second edit box: This is a description field. You can write the sub composition description here,
* Connector List: This is the list of all sub composition connectors. The name field is editable,
* Apply button: Click on this button to apply your change,
* Cancel button: Click on this button to discard your change.

Error

back to top

Highlighted

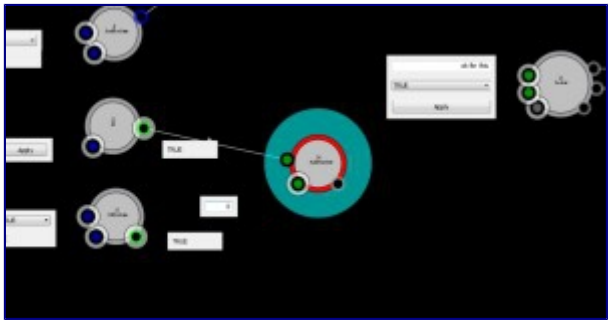


Figure 51 – Highlighted

When an error occurs, Caméléon highlight in red erroneous components.

Browse error

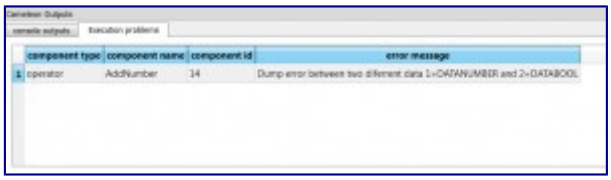


Figure 52 – Error list

Document édité sous licence **Creative Common**

You can browse error using the Execution problems tab on Caméléon Outputs (bottom part of the composer).
Double click on an error id show on the composer or controller the component where the error occurs.

Lock pop-up

Figure 53 – Lock pop-up

At stop or pause, a Lock pop-up may appears when a component has not yet finished its work. You can press the save & restart button to relaunch Caméléon. It may be useful in debug mode when components has some infinite execution problems.

Command Line Interface (CLI)

back to top

In a command line (use git bash on windows), in the instance/bin directory, you can launch a composition like this:

```
./cameleon-cli.exe -mode="CLI" -cm="../project/yourProject.cm"
```

The help menu can be displayed with:

```
./cameleon-cli.exe -h
```

```
NAME
        Cameleon program launcher
SYNOPSIS
        ide.exe [options]
DESCRIPTION
        If no option is specified, it launch the .cm file specified in the
Caméléon instance file
        -h:      print this help or print the .cm file help if -cm option is
specified
        -cm=[.cm file path]:     launch a specific .cm file
        -mode=[IDE, GUI, CLI]:   set the Caméléon mode
                IDE:     launch Caméléon creator in IDE mode,
                GUI:     launch Caméléon app with the related GUI interface,
                CLI:     launch Caméléon in a command line mode,
EXAMPLES
        Standard launch with full IDE    cameleon.exe
        See the help of a specified .cm file     cameleon.exe -h
-cm="../project/tutorial/example-0XY-Arguments/example-0XY-Arguments.cm"
        Launch a specified .cm file with full IDE        cameleon.exe
-cm="../project/tutorial/example-0XY-Arguments/example-0XY-Arguments.cm"
        Launch a specified .cm file in command line application mode
cameleon.exe -mode="CMD" -cm="../project/tutorial/example-0XY-Arguments/example-
0XY-Arguments.cm"
        Launch a specified .cm file in GUI application mode      cameleon.exe
-mode="GUI" -cm="../project/tutorial/example-0XY-Arguments/example-0XY-
```

```
Arguments.cm"
```

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org



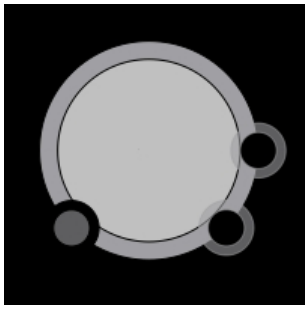
Standard Caméléon Dictionary Reference

Operators

[Boolean](#)
[Number](#)
[String](#)
[Vector](#)
[Table](#)
[Map](#)
[Generic](#)
[Control-Flow](#)
[FileSystem](#)
[Introspection](#)
[Test](#)

Controls

[FileSystem](#)
[Boolean](#)
[Number](#)
[String](#)
[Vector](#)
[Map](#)
[Table](#)
[Cli](#)
[Interaction](#)
[Introspection](#)



Copy

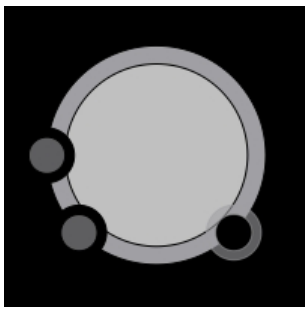
Inputs: in.gen (CDATAGENERIC)
Outputs: copy0.gen (CDATAGENERIC) copy1.gen (CDATAGENERIC)

Description

Copy the single input data to the output datas

Boolean (Back to top)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

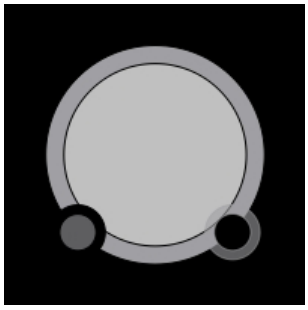


AndBoolean

Inputs: in1.bool (DATABOOL) in2.bool (DATABOOL)
Outputs: out.bool (DATABOOL)

Description

out= true for in1=true and in2=true, false otherwise

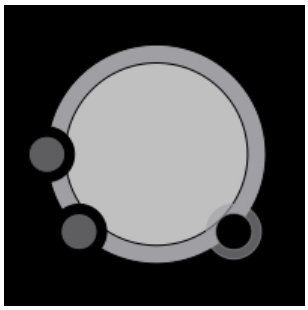


InverseBoolean

Inputs: in.bool (DATABOOL)
Outputs: out.bool (DATABOOL)

Description

out= true for in1=false, false otherwise



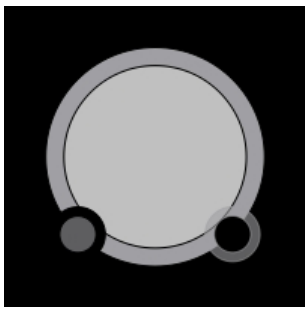
OrBoolean

Inputs: in1.bool (DATABOOL) in2.bool (DATABOOL)
Outputs: out.bool (DATABOOL)

Description

out= true for in1=true or in2=true, false otherwise

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

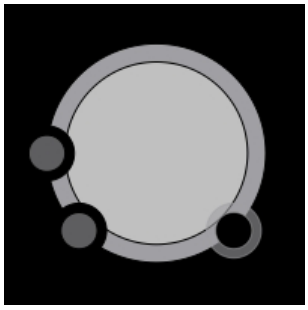


LoadBoolean

Inputs: file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Load the boolean value in the given file



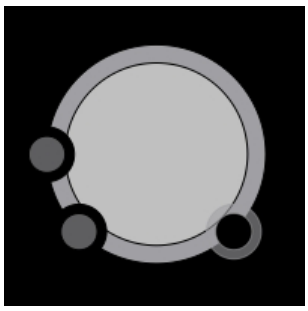
SaveBoolean

Inputs: in.bool (DATABOOL) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the boolean value in the given file, out= false for bad writing, true otherwise

Number (Back to top)



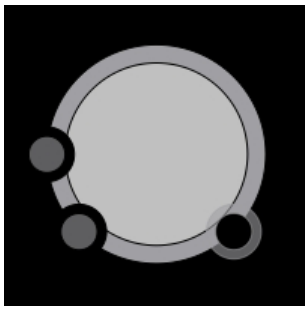
AddNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

Document édité sous licence **Creative Common**

v=v1+v2

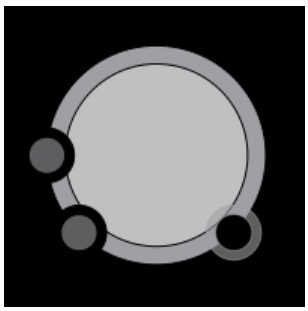


SubNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

v=v1-v2

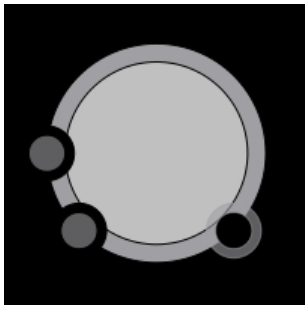


DivNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

v=v1/v2



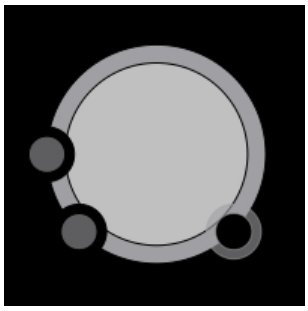
EqualNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: out.bool (DATABOOL)

Description

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

out = true for v1=v2, false otherwise

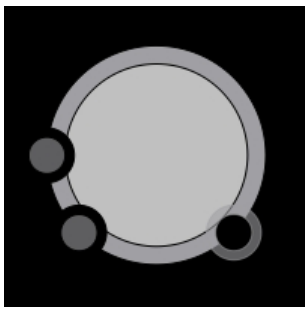


InfNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: out.bool (DATABOOL)

Description

out = true for v1″<"v2, false otherwise

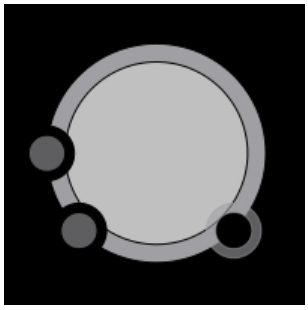


MultNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

v=v1*v2

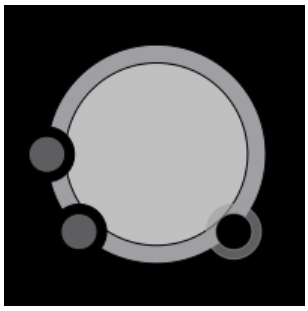


SupNumber

Inputs: a.num (DATANUMBER) b.num (DATANUMBER)
Outputs: out.bool (DATABOOL)

Description

out = true for a”>”b, false otherwise

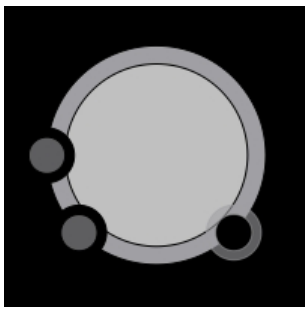


MaxNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

v=max(v1,v2)

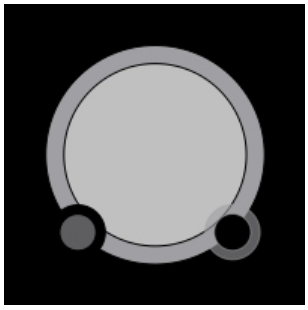


MinNumber

Inputs: v1.num (DATANUMBER) v2.num (DATANUMBER)
Outputs: v.num (DATANUMBER)

Description

v=Min(v1,v2)

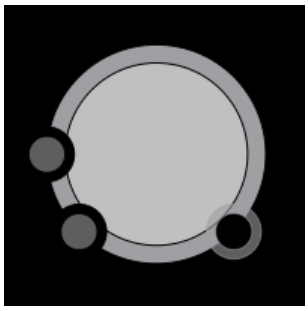


AbsNumber

Inputs: vin.num (DATANUMBER)
Outputs: vout.num (DATANUMBER)

Description

vout = abs(vin)

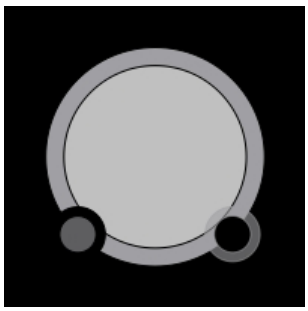


PowNumber

Inputs: v.num (DATANUMBER) n.num (DATANUMBER)
Outputs: x.num (DATANUMBER)

Description

x=v^n

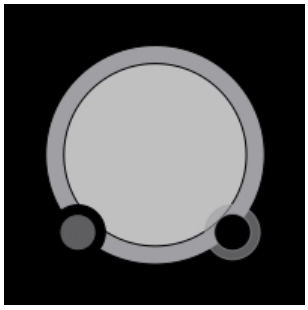


FromStringNumber

Inputs: in1.str (DATASTRING)
Outputs: result.num (DATANUMBER)

Description

” No



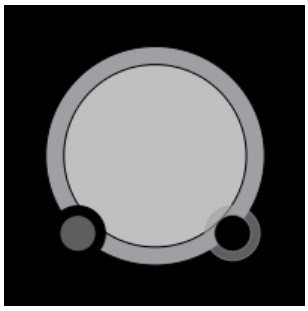
ToStringNumber

Inputs: in.num (DATANUMBER)
Outputs: result.str (DATASTRING)

Description

Document édité sous licence **Creative Common**

” No

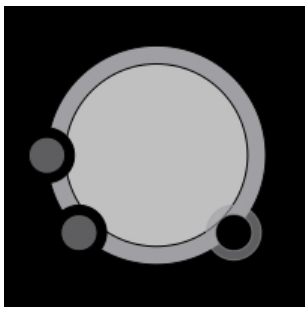


LoadNumber

Inputs: file.str (DATASTRING)
Outputs: out.bool (DATANUMBER)

Description

Load the number value in the given file



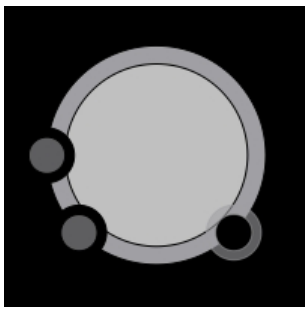
SaveNumber

Inputs: in.bool (DATANUMBER) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the Number value in the given file, out= false for bad writing, true otherwise

String (Back to top)



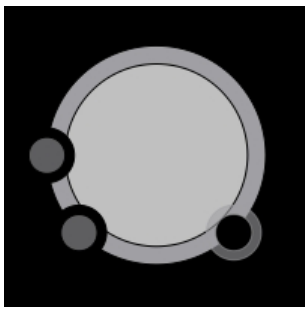
ConcatString

Inputs: in1.str (DATASTRING) in2.str (DATASTRING)
Outputs: out.str (DATASTRING)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

out=[in1,in2]

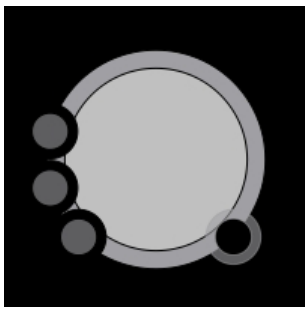


SplitRegExpString

Inputs: string.str (DATASTRING) regexp.str (DATASTRING)
Outputs: v_out.str (DATAVECTOR)

Description

Splits the string into substrings wherever the regular expression regexp matches
The substrings are include in a vector of DataString

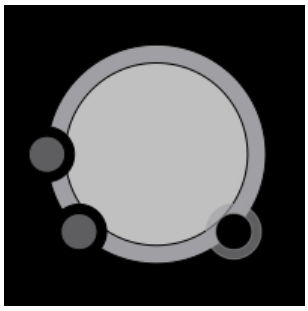


ReplaceString

Inputs: text.str (DATASTRING) before.str (DATASTRING) after.str (DATASTRING)
Outputs: out.str (DATASTRING)

Description

Replaces every occurrence of the string before with the string after in the string text



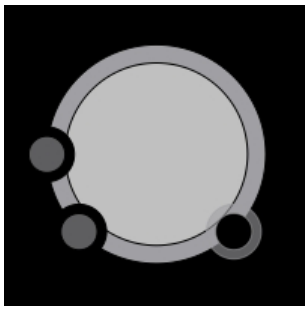
SplitString

Inputs: in1.str (DATASTRING) separtor.str (DATASTRING)
Outputs: string.vec (DATAVECTOR)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

Splits the string into substrings with the separtor string
The substrings are included in a vector of DataString

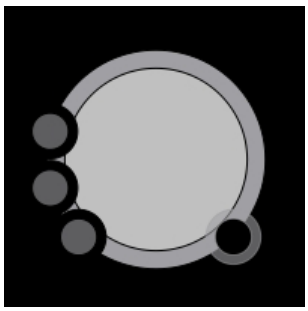


CompareString

Inputs: in1.str (DATASTRING) in2.str (DATASTRING)
Outputs: compare.bool (DATABOOL)

Description

Compare=true for in1=in2, false otherwise

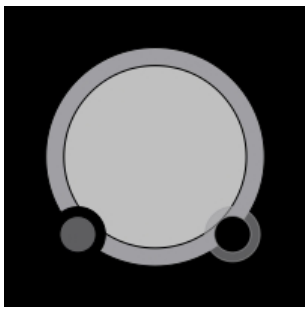


InsertString

Inputs: in.str (DATASTRING) in.num (DATANUMBER) str.str (DATASTRING)
Outputs: out.num (DATASTRING)

Description

The current string content is extended by inserting some additional content at a specific location within the string content.



SizeString

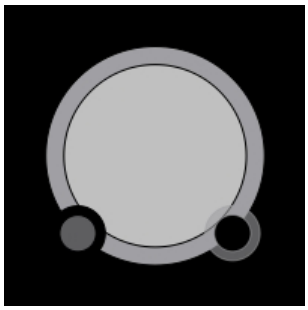
Inputs: in.str (DATASTRING)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Outputs: out.num (DATANUMBER)

Description

Size of the input string

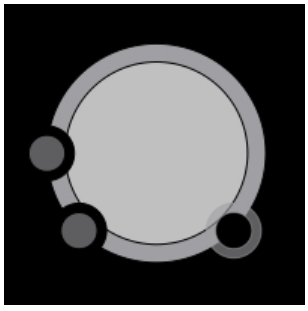


LoadString

Inputs: file.str (DATASTRING)
Outputs: in.str (DATASTRING)

Description

” No



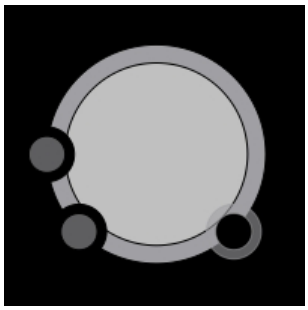
SaveString

Inputs: in.str (DATASTRING) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the String value in the given file, out= false for bad writing, true otherwise

Vector (Back to top)



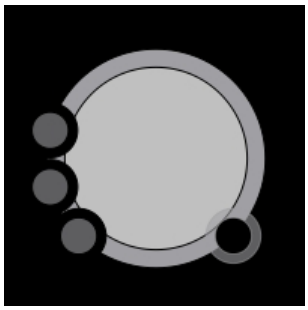
Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

GetElementVector

Inputs: in.vec (DATAVECTOR) index.num (DATANUMBER)
Outputs: element.gen (CDATAGENERIC)

Description

Get the data given by the index

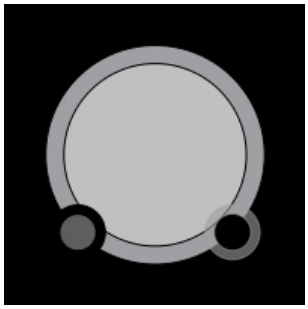


SetDataVector

Inputs: in.vec (DATAVECTOR) index.num (DATANUMBER) element.gen (CDATAGENERIC)
Outputs: out.vec (DATAVECTOR)

Description

Set the data element of the vector at the given index

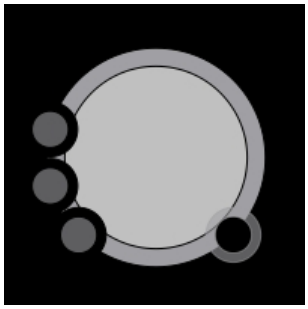


SizeVector

Inputs: in.vec (DATAVECTOR)
Outputs: size.num (DATANUMBER)

Description

Size of the input vector



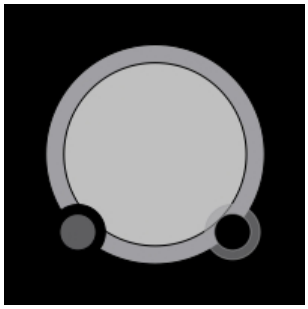
Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

EraseVector

Inputs: in.vec (DATAVECTOR) index1.num (DATANUMBER) index2.num (DATANUMBER)
Outputs: out.vec (DATAVECTOR)

Description

removes from the vector container either a single element (index1) or a range of elements ([index1,index2)).

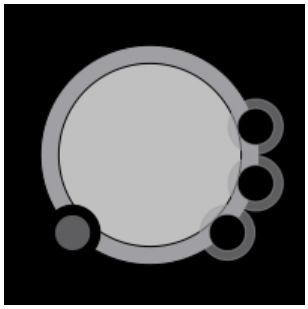


SortVector

Inputs: in.vec (DATAVECTOR)
Outputs: out.vec (DATAVECTOR)

Description

Sort the input vector following the member toString of the data



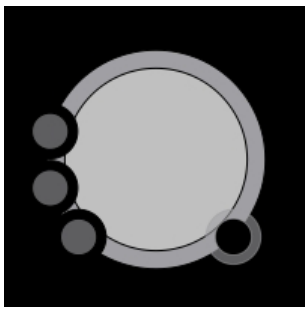
IteratorVectorIn

Inputs: in.vec (DATAVECTOR)
Outputs: out.vec (DATAVECTOR) data.out (CDATAGENERIC) index.num (DATANUMBER)

Description

Iterate over the vector from begin to end every time a NEW . During this itration, boolean state is EMPTY and at the end is NEW with a true value

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

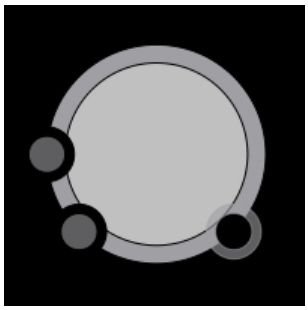


IteratorVectorOut

Inputs: in.vec (DATAVECTOR) index.num (DATANUMBER) data.out (CDATAGENERIC)
Outputs: out.vec (DATAVECTOR)

Description

Iterate over the vector from begin to end every time a NEW . During this itration, boolean state is EMPTY and at the end is NEW with a true value

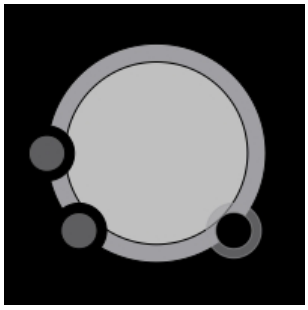


PushBackVector

Inputs: in.vec (DATAVECTOR) data.gen (CDATAGENERIC)
Outputs: out.vec (DATAVECTOR)

Description

Push back data in the vector



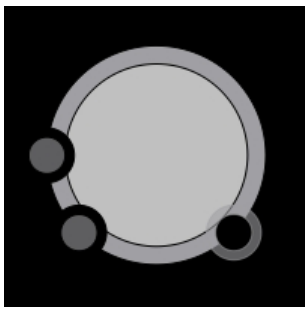
ResizeVector

Inputs: in.vec (DATAVECTOR) index.num (DATANUMBER)
Outputs: out.vec (DATAVECTOR)

Description

Set the data element of the vector at the given index

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

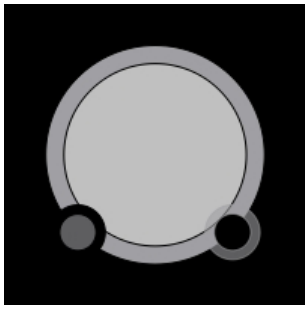


SaveVector

Inputs: in.vec (DATAVECTOR) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the Vector value in the given file, out= false for bad writing, true otherwise



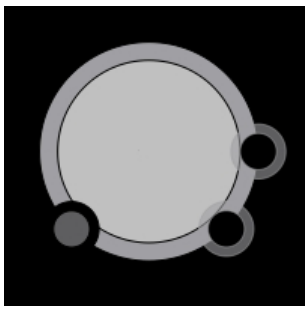
LoadVector

Inputs: file.str (DATASTRING)
Outputs: m.vec (DATAVECTOR)

Description

Load the Vector in the given file

Table (Back to top)



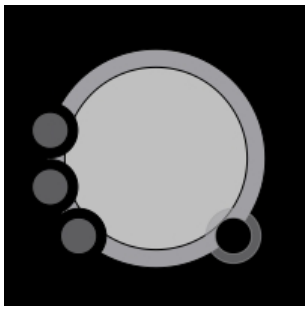
SizeTable

Inputs: table.tab (DataTable)
Outputs: colsize.num (DATANUMBER) rowsize.num (DATANUMBER)

Description

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Get the size of the input table

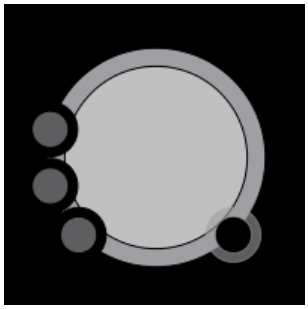


ResizeTable

Inputs: tablein.tab (DataTable) colsize.num (DATANUMBER) rowsize.num (DATANUMBER)
Outputs: tableout.tab (DataTable)

Description

Resize the input table

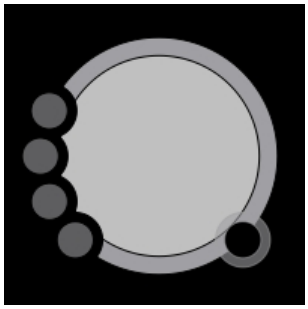


GetValueTable

Inputs: tablein.tab (DataTable) col.num (DATANUMBER) row.num (DATANUMBER)
Outputs: elt.str (DATASTRING)

Description

get the element at the given position row and col



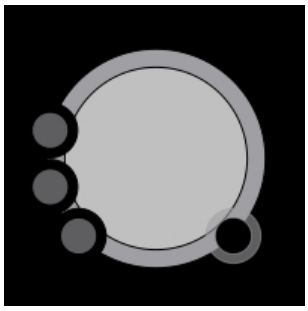
SetValueTable

Inputs: tablein.tab (DataTable) col.num (DATANUMBER) row.num (DATANUMBER) elt.str (DATASTRING)
Outputs: tableout.tab (DataTable)

Description

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

set the value at the given position row col

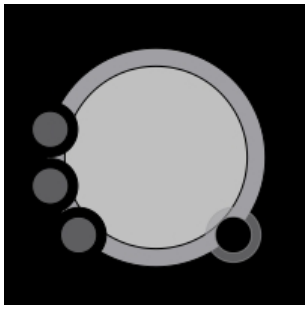


GetColTable

Inputs: tablein.tab (DataTable) colmin.num (DATANUMBER) colmax.num (DATANUMBER)
Outputs: tableout.tab (DataTable)

Description

Extract the table between colmin and colmax with colmax=colmin as default value

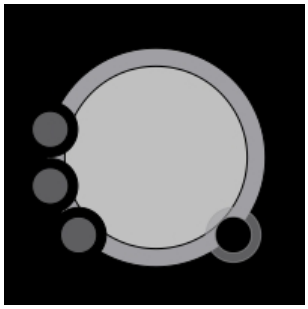


GetRowTable

Inputs: tablein.tab (DataTable) Rowmin.num (DATANUMBER) Rowmax.num (DATANUMBER)
Outputs: tableout.tab (DataTable)

Description

Extract the table between Rowmin and Rowmax with Rowmax=Rowmin as default value



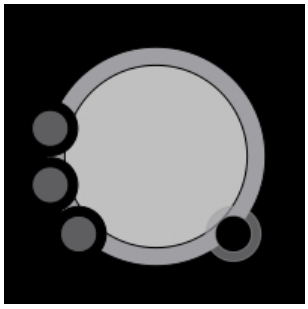
RmColTable

Inputs: tablein.tab (DataTable) Colmin.num (DATANUMBER) Colmax.num (DATANUMBER)
Outputs: tableout.tab (DataTable)

Description

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Remove the columns of the input table the between Colmin and Colmax with Colmax=Colmin as default value

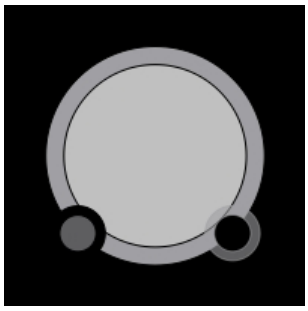


RmRowTable

Inputs: tablein.tab (DataTable) Rawmin.num (DATANUMBER) Rawmax.num (DATANUMBER)
Outputs: tableout.tab (DataTable)

Description

Remove the Rawumns of the input table the between Rawmin and Rawmax with Rawmax=Rawmin as default value

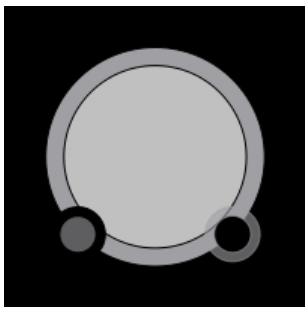


CSVToTable

Inputs: csv.str (DATASTRING)
Outputs: m.Table (DataTable)

Description

Load the Table from the given csv format string



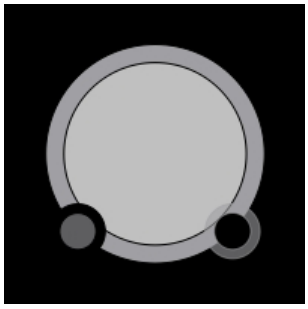
TableToCSV

Inputs: in.tab (DataTable)
Outputs: csv.str (DATASTRING)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

Return a string representing the full text CSV file

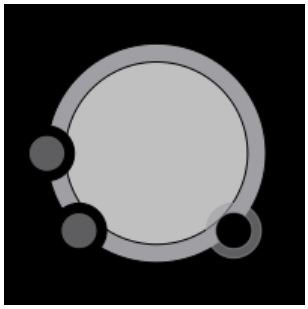


LoadTable

Inputs: file.str (DATASTRING)
Outputs: m.Table (DataTable)

Description

Load the Table in the given file



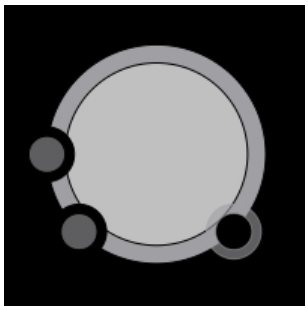
SaveTable

Inputs: in.tab (DataTable) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the Table value in the given file, out= false for bad writing, true otherwise

Map (Back to top)



GetDataMap

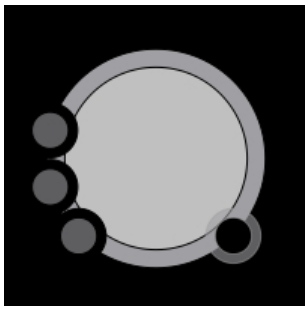
Inputs: in1.map (DATAMAP) key.str (DATASTRING)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Outputs: value.gen (CDATAGENERIC)

Description

Get the data given by the key

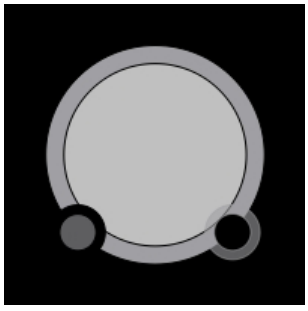


InsertMap

Inputs: min.map (DATAMAP) key.str (DATASTRING) dataelt.gen (CDATAGENERIC)
Outputs: mout.map (DATAMAP)

Description

Insert the element data in the map with the given key

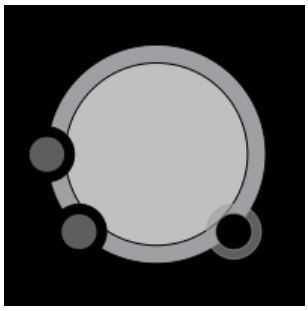


KeysMap

Inputs: m.map (DATAMAP)
Outputs: keys.vec (DATAVECTOR)

Description

Create a vector containing DataString elements with the keys of the input map



RemoveMap

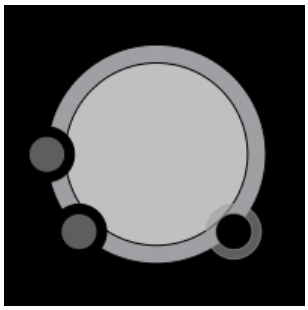
Inputs: min.map (DATAMAP) key.str (DATASTRING)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Outputs: mout.map (DATAMAP)

Description

Remove the element with the given key

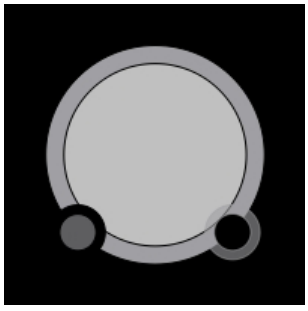


SaveMap

Inputs: in.bool (DATAMAP) file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Save the Map value in the given file, out= false for bad writing, true otherwise



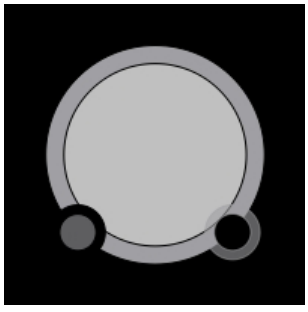
LoadMap

Inputs: file.str (DATASTRING)
Outputs: m.map (DATAMAP)

Description

Load the map in the given file

Generic (Back to top)



Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

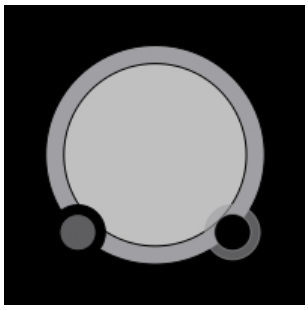
Blank

Inputs: datatype.str (DATASTRING)
Outputs: blank.gen (CDATAGENERIC)

Description

Create a blank data following the data type

Control-Flow (Back to top)

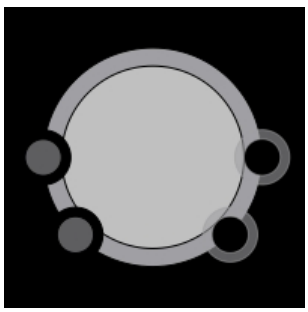


Id

Inputs: input0.gen (CDATAGENERIC)
Outputs: output0.gen (CDATAGENERIC)

Description

Copy the input datas of the output datas of the same index starting from the lowest index to the highest



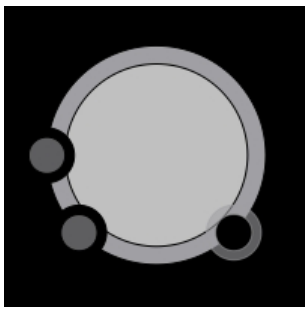
If

Inputs: boolean.bool (DATABOOL) input.gen (CDATAGENERIC)
Outputs: if.gen (CDATAGENERIC) else.gen (CDATAGENERIC)

Description

Copy input to if for boolean=true or Copy input to else for boolean=false

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

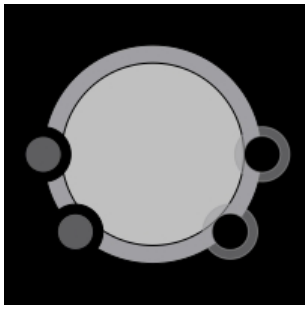


Merge

Inputs: input0.gen (CDATAGENERIC) input1.gen (CDATAGENERIC)
Outputs: output.gen (CDATAGENERIC)

Description

Copy input data at new to output data.
"<"b">"Execution Condition:"<"/b">"
* one input at new,
* output to OLD.
Action:
* can add input connector.

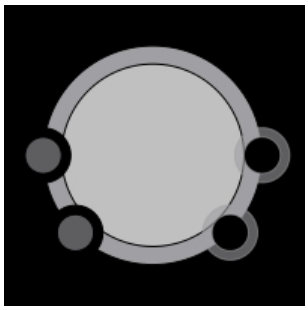


Synchrone

Inputs: input0.gen (CDATAGENERIC) input1.gen (CDATAGENERIC)
Outputs: output0.gen (CDATAGENERIC) output1.gen (CDATAGENERIC)

Description

Exec if all input plug states are NEW



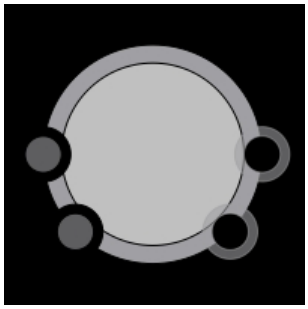
Asynchrone

Inputs: input1.gen (CDATAGENERIC) input2.gen (CDATAGENERIC)
Outputs: output1.gen (CDATAGENERIC) output2.gen (CDATAGENERIC)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

Exec if at least one input plug state is NEW

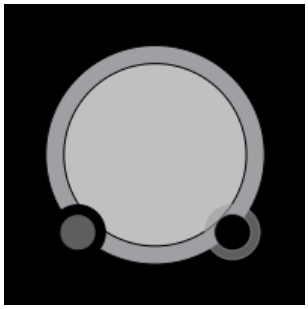


Locker

Inputs: lockin.gen (CDATAGENERIC) in0.gen (CDATAGENERIC)
Outputs: lockout.gen (CDATAGENERIC) out0.gen (CDATAGENERIC)

Description

Can be executed if lock state=NEW
the exex is just the copy of the data and the state from in_i to out_i

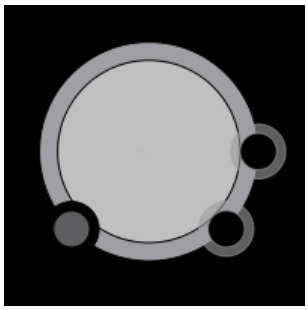


Erase

Inputs: input1.gen (CDATAGENERIC)
Outputs: output1.gen (CDATAGENERIC)

Description

input1 to output1 if input1 == NEW & whatever the state of the output plug.



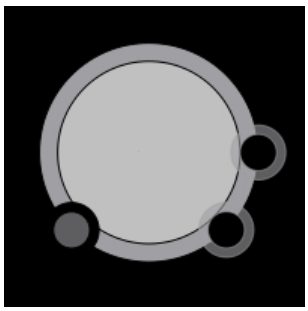
Counter

Inputs: input.gen (CDATAGENERIC)
Outputs: output.gen (CDATAGENERIC) count.num (DATANUMBER)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

Count the number of execution of this operator for the beginning of play mode

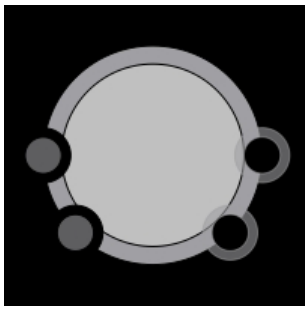


Loop

Inputs: max.num (DATANUMBER)
Outputs: it.num (DATANUMBER) end.bool (DATABOOL)

Description

for(int it=0;it"<"max;it++), max.num is OLD and end = true when it==max

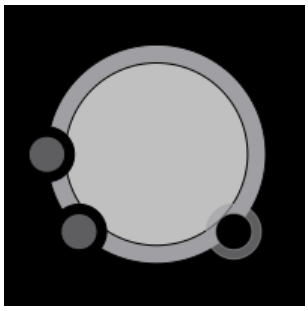


forteleport

Inputs: nbrloop.num (DATANUMBER) data_0_iteration_0.gen (CDATAGENERIC)
Outputs: connectionwithforloopend.num (DATANUMBER) data_0_iteration_i.gen (CDATAGENERIC)

Description

” No



endforteleport

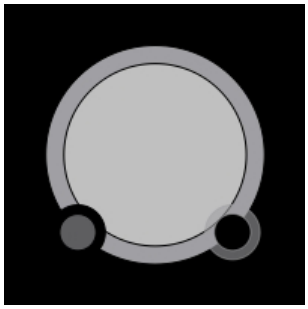
Inputs: connectionwithforloop.num (DATANUMBER) data_0_iteration_i.gen (CDATAGENERIC)
Outputs: data_0_iteration_exitloop.gen (CDATAGENERIC)

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Description

” No

FileSystem (Back to top)

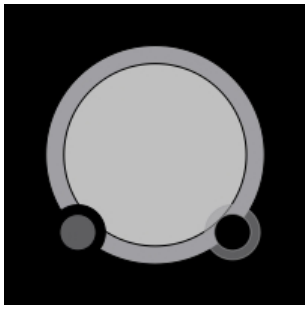


IsExist

Inputs: path.str (DATASTRING)
Outputs: exists.bool (DATABOOL)

Description

exists=true if the path exist, false otherwise

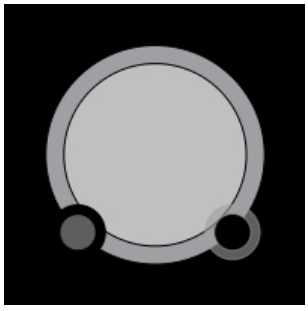


IsDir

Inputs: path.str (DATASTRING)
Outputs: isDir.bool (DATABOOL)

Description

isDir=true if the path is a directory, false otherwise



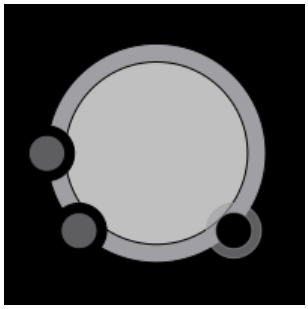
IsFile

Document édité sous licence **Creative Common**

Inputs: path.str (DATASTRING)
Outputs: isFile.bool (DATABOOL)

Description

isFile=true if the path is a file, false otherwise

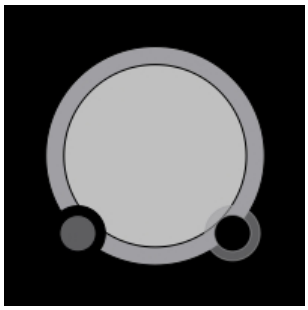


ReadDir

Inputs: dirpath.str (DATASTRING) recurcive.bool (by default false) (DATABOOL)
Outputs: string.vec (DATAVECTOR)

Description

List every entries (file and directory) from an absolute dir path.

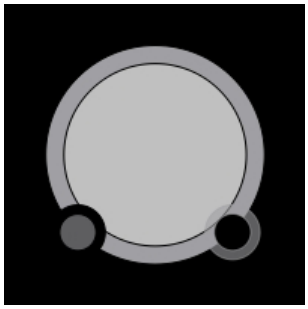


Readfile

Inputs: file path.str (DATASTRING)
Outputs: content.str (DATASTRING)

Description

” No



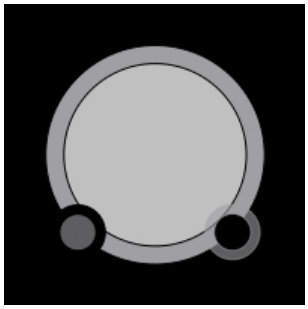
AbsoluteDir

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

Inputs: path.str (DATASTRING)
Outputs: abspath.str (DATASTRING)

Description

Get the absolute directory (by removing the file name in case of file)

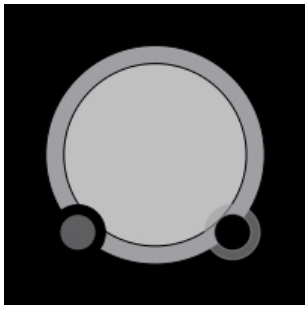


BaseName

Inputs: path.str (DATASTRING)
Outputs: basename.str (DATASTRING)

Description

Get the base name of the file without the path (for instance path=/home/vincent/lena.pgm -”>” basename=lena

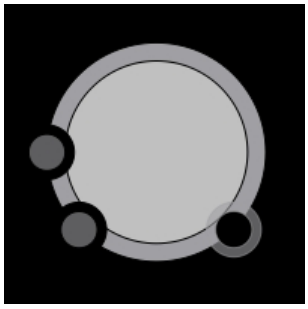


Suffix

Inputs: path.str (DATASTRING)
Outputs: Suffix.str (DATASTRING)

Description

Get the suffix of the file without the path without including the dot (for instance path=/home/vincent/lena.pgm -”>” Suffix=pgm



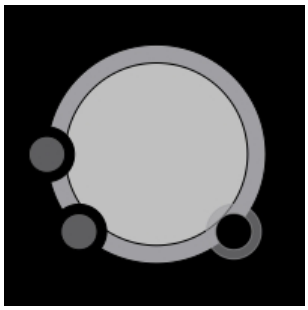
Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

CopyDir

Inputs: source.str (DATASTRING) target.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Copy source path dir content to target path dir

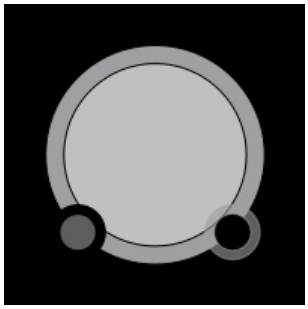


CopyFile

Inputs: file1.str (DATASTRING) file2.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

copy file1 to file2

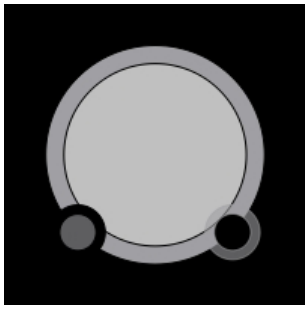


RmDir

Inputs: dir.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

remove dir



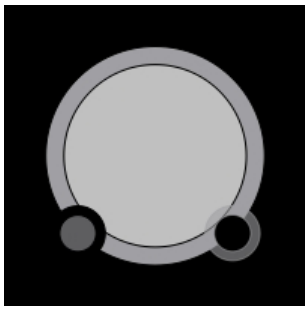
Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org

RmFile

Inputs: file.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

remove file



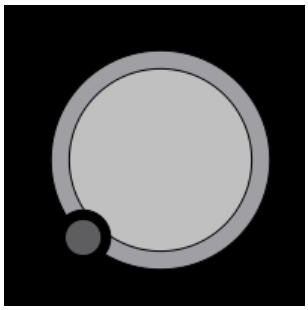
MakeDir

Inputs: target.str (DATASTRING)
Outputs: out.bool (DATABOOL)

Description

Create a directory

Introspection (Back to top)



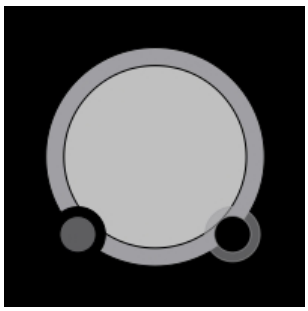
ExitInstance

Inputs: exit.bool (DATABOOL)
Outputs:

Description

exit & kill current Caméléon instance

Document édité sous licence **Creative Common**



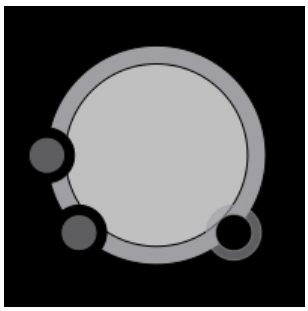
BreakPoint

Inputs: input0.gen (CDATAGENERIC)
Outputs: output0.gen (CDATAGENERIC)

Description

pause & next the current Caméléon instance

Test (Back to top)



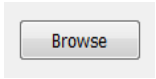
Assert

Inputs: test.bool (DATABOOL) name.str (DATASTRING)
Outputs: test.bool (DATABOOL)

Description

” No

FileSystem (Back to top)



OpenFile

Inputs: out.vec (DATAVECTOR) name.str (DATASTRING) record.bool (DATABOOL)
Outputs: out.str (DATASTRING)

Description

” No

Document édité sous licence **Creative Common**



SaveFile

Inputs: out.vec (DATAVECTOR) name.str (DATASTRING) record.bool (DATABOOL)
Outputs: out.str (DATASTRING)

Description

” No



OpenDirectory

Inputs: name.str (DATASTRING) record.bool (DATABOOL)
Outputs: out.str (DATASTRING)

Description

” No



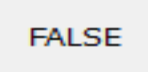
SelectorFile

Inputs:
Outputs: path.str (DATASTRING)

Description

” No

Boolean (Back to top)

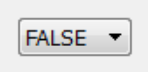


ViewBoolean

Inputs: in.bool (DATABOOL)
Outputs:

Description

” No



EditorBoolean

Inputs:

Document édité sous licence **Creative Common**

Outputs: out.bool (DATABOOL)

Description

” No



Button

Inputs: name.str (DATASTRING)
Outputs: clicked.bool (DATABOOL)

Description

Set the boolean



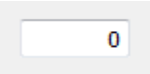
EditorCheckBox

Inputs: default.bool (DATABOOL)
Outputs: out.bool (DATABOOL)

Description

” No

Number (Back to top)

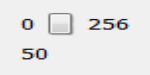


EditorNumber

Inputs:
Outputs: out.num (DATANUMBER)

Description

Press enter to set the value



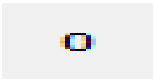
SliderNumber

Inputs: min.num (DATANUMBER) max.num (DATANUMBER) step.num (DATANUMBER)
Outputs: out.num (DATANUMBER)

Description

” No

Document édité sous licence **Creative Common**



ViewNumber

Inputs: in.num (DATANUMBER)
Outputs:

Description

” No



ProgressBar

Inputs: min.num (DATANUMBER) max.num (DATANUMBER) current.num (DATANUMBER)
Outputs:

Description

Progress bar defined by a range [min,max] and a current value

String (Back to top)

ViewStringText

Inputs: in.str (DATASTRING)
Outputs:

Description

” No

EditorStringLine

Inputs:
Outputs: out.str (DATASTRING)

Description

” No

EditorStringText

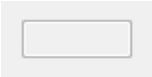
Inputs:

Document édité sous licence **Creative Common**

Outputs: out.str (DATASTRING)

Description

” No



ViewStringLine

Inputs: in.str (DATASTRING)
Outputs:

Description

” No

EditorStringLine(input)

Inputs: init.str (DATASTRING)
Outputs: out.str (DATASTRING)

Description

” No

Vector (Back to top)

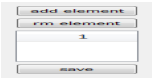


ListBoxVector

Inputs: in.vec (DATAVECTOR)
Outputs: data.str (DATASTRING) type.str (DATASTRING)

Description

” No



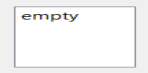
EditorVector

Inputs:
Outputs: out.vec (DATAVECTOR)

Description

” No

Document édité sous licence **Creative Common**



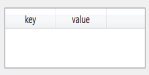
ViewVector

Inputs: in.vec (DATAVECTOR)
Outputs:

Description

” No

Map (Back to top)

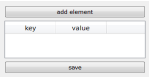


ViewMap

Inputs: in.map (DATAMAP)
Outputs:

Description

” No



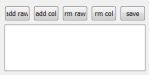
EditorMap

Inputs:
Outputs: out.map (DATAMAP)

Description

” No

Table (Back to top)



EditorTable

Inputs:
Outputs: out.table (DataTable)

Description

” No



Document édité sous licence **Creative Common**

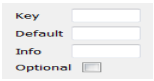
ViewTable

Inputs: in.table (DataTable)
Outputs:

Description

” No

Cli (Back to top)



Argument

Inputs:
Outputs: out.str (DATASTRING)

Description

” No

Interaction (Back to top)

ViewMenu

Inputs: description.tab (DataTable)
Outputs: key.str (DATASTRING)

Description

” No

Introspection (Back to top)

Background

Inputs:
Outputs:

Description

” No

Caméléon (2.0.13) is an open source software edited by shinoe,
designed & developed by O. Cugnon de Sevricourt & V. Tariel

Document édité sous licence **Creative Common**
0.0.10 29/10/2016
http://www.shinoe.org





Document édité sous licence **Creative Common**

0.0.10 29/10/2016

http://www.shinoe.org

